

SoK: On the Lattice-based Constructions of Universal Thresholdizer

Navid Abapour
Surrey Centre for Cyber Security
University of Surrey
n.abapour@surrey.ac.uk

Constantin Cătălin Drăgan
Surrey Centre for Cyber Security
University of Surrey
c.dragan@surrey.ac.uk

Abstract—To be written...

I. INTRODUCTION

The threshold cryptography originates from the work on threshold secret sharing by Shamir [1] and Blakley [2] in the late 1970s. These schemes addressed the problem of partitioning a secret among multiple parties such that a specified threshold of participants is required for reconstruction, and subsequent research extended these principles to specific cryptographic primitives. These constructions, however, required distinct designs for each primitive, resulting in a distributed landscape of threshold protocols with heterogeneous security properties and efficiency parameters.

To address this drawback, *Universal Thresholdizers* (UT) were first introduced by Boneh et al [3] as a blackbox method to realize threshold primitives from non-threshold primitives, e.g., CCA-secure threshold public key encryption, and have progressed since their introduction as a framework for transforming standard primitives into threshold variants [4]. This is achieved by reinterpreting the core operation of those primitives (e.g., signing or decryption algorithm) as a monotone boolean circuit and using UT to produce partial evaluations - the circuit evaluated on shares of the secret key. *Correctness* of UT ensures that combining the outcomes of partial evaluations produces the same output as evaluating the circuit directly on the secret key. This is strengthened by *simulation security* for UT, where the adversary can not distinguish between real partial evaluations and fake partial evaluations (i.e., the circuit outcome is simulated by a trusted 3rd party). Additionally, UT is *compact* - where the size of the partial evaluation is linear in the size of the circuit depth and number of shares [3].

Universal Thresholdizers provide a unifying framework that addresses aforementioned heterogeneity through a standardized methodology for transforming conventional primitives into threshold variants [5]. The UT comprises algorithms for setup, evaluation, verification, and combination that facilitate the distributed computation of functions representable as circuits. The security model of UT encompasses two principal properties: *simulation security*, which establishes that partial evaluations disclose no information beyond the function output, and *robustness*, which ensures that adversarial participants

cannot generate invalid partial evaluations that verification would accept [3], [4].

The implementation of the UT paradigm with optimal efficiency while preserving both security and robustness remains a complex problem. The original construction by Boneh et al. [4] employed Threshold Fully Homomorphic Encryption (TFHE) in conjunction with zero-knowledge proofs with preprocessing (PZK) and commitment schemes (Com). This approach, while theoretically rigorous, incurs computational overhead, particularly for complex threshold access structures. These constraints have motivated the development of alternative constructions with different trade-offs between security, robustness, and efficiency.

Campanelli and Khoshakhlagh [6] proposed two distinct approaches to UT construction: one based on the combination of TFHE with Homomorphic Signatures (HS), and another leveraging Homomorphic Secret Sharing (HSS) with HS for two-party threshold settings. While these constructions offer computational advantages compared to the original proposal, our analysis identifies a fundamental incompatibility between security and robustness in signature-based approaches that previous literature has not formalized.

More recently, Ebrahimi and Yadav [5] introduced a construction based on TFHE with Key-Homomorphic Pseudo-random Functions (KHPRF), with the objective of enhancing the security guarantees of Universal Thresholdizers. Concurrently, Cheon et al. [7] proposed a communication-efficient construction using Tree-based Secret Sharing (TreeSSS), which reduces the asymptotic number of secret shares required from $O(N^{5.3})$ to $O(N^{3+o(1)})$ for N participants.

Given these diverse approaches, Universal Thresholdizer requires a systematic analysis of the trade-offs between different construction methods for theoretical advancement. To our knowledge, as of today, there has been no systematic study or comprehensive literature review on threshold cryptography, and considering the concept of universal thresholdizer has been a constructive step towards achieving a general framework for threshold cryptography, in this work, we initiate it with universal thresholdizers.

A. Contributions

This work's approach is characterized by several questions:

- To what extent are simulation security and robustness compatible, and what are the inherent trade-offs observed in certain construction methods (Security-Robustness Relationship)?
- How do various construction approaches differ in terms of quantifiable computational and communication complexity, and what are the implications for practical implementations (Efficiency Metrics)?
- Are there comprehensive formalisms for comparing different UT constructions and analyzing their properties?
- Is there a more straightforward way to achieve UT (Dynamic Model)? What is the formal distinction between static and dynamic UT, and how does this distinction impact their security and applicability (Dynamic vs. Static Models)?

This paper presents a structured evaluation of UT constructions, providing a comprehensive taxonomy of design methodologies, security properties, and efficiency parameters. We identify precise distinctions between construction methods with implications for practical implementations. Our contributions include:

- A refined formalization of UT that differentiates between static and dynamic variants, and establishes the relationship between them through a constructive reduction.
- Identification and proof of an incompatibility between simulation security and robustness in signature-based UT constructions, establishing that these properties cannot be simultaneously achieved.
- A formal analysis of five distinct construction approaches for UT, examining their foundations, security guarantees, and applicability to different threshold settings.
- The quantitative efficiency analysis of each construction method, computing time and space complexity for setup, evaluation, verification, and combination operations.

Our study establishes that existing UT constructions do not provide optimal performance across all dimensions of security, robustness, and efficiency, e.g., the construction of Boneh et al. [4] achieves both security and robustness at the cost of increased computational requirements, while the signature-based approach of Campanelli and Khoshakhlagh [6] offers reduced computational complexity but cannot simultaneously achieve both security properties. The TreeSSS-based construction of Cheon et al. [7] provides superior communication efficiency for large committees, while the two-party construction using HSS is optimized for settings with exactly two participants.

B. Overview of Paper

The remainder of this work is structured as follows. Section II establishes the notation necessary for our analysis. Section III formalizes the framework of UT, distinguishing between static and dynamic variants. Section IV presents a comprehensive taxonomy of construction techniques, analyzing five distinct approaches to UT. Section V proves the fundamental incompatibility between simulation security and robustness for signature-based constructions. Section VI

provides a quantitative efficiency analysis across all construction methods, computing computational complexity for key operations.

II. PRELIMINARIES

Here, we provide some background on building blocks and introduce the most commonly used notation that we utilize throughout this work; the rest is detailed in Appendix A.

Learning with Errors (LWE)

Let $n, m, q \in \mathbb{N}$, and let χ be a distribution over $\mathbb{Z}_q$. Consider $D_0 := (\mathbf{A}, \mathbf{A}^T \mathbf{s} + \mathbf{e})$ and $D_1 := (\mathbf{A}, \mathbf{u})$, where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$, $\mathbf{s} \leftarrow \mathbb{Z}_q^n$, $\mathbf{e} \leftarrow \chi^m$, $\mathbf{u} \leftarrow \mathbb{Z}_q^m$. The LWE(n, m, q, χ) problem is to decide between the distributions D_0 and D_1 [8]. Furthermore, if the error distribution χ is bounded by a parameter B_{lwe} (i.e., every sample from χ has an absolute value at most B_{lwe}), then solving LWE(n, m, q, χ) is at least as hard as approximating worst-case problems such as GapSVP and SIVP [9] in n dimensions within the factor of $\tilde{O}(n \cdot q / B_{\text{lwe}})$.

Short Integer Solution (SIS)

Let $n, m, q \in \mathbb{N}$ and let $\beta > 0$. Define SIS(n, m, q, β) as the problem of finding a non-zero vector $\mathbf{z} \in \mathbb{Z}^m$ such that $\mathbf{A}\mathbf{z} = \mathbf{0} \pmod{q}$ and $\|\mathbf{z}\| \leq \beta$, where $\mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}$ [10].

Efficient Access Structures ($\mathbb{S}, N$)

We consider *access structures* as all sets of authorized parties $\mathbb{A} \subset \mathbb{S}$ with N total parties $|\mathbb{A}| \leq N$. Access structures are *efficient* if they are monotone (i.e., $\forall \mathbb{A} \subset \mathbb{S}$ and $|\mathbb{A}| < N$ implies $\mathbb{A} \cup \{x\} \subset \mathbb{S}$) and they can be described using a boolean circuit $f_{\mathbb{A}} : \{0, 1\}^N \rightarrow \{0, 1\}$. A particular type of efficient access structures is *threshold access structures* where a threshold t defines the minimal size of the authorized set: $t \leq |\mathbb{A}|$ for $\mathbb{A} \subset \mathbb{S}$ [4].

Circuit class ($\mathbb{C}, d$)

We use circuits of arbitrary inputs $C : \{0, 1\}^k \rightarrow \{0, 1\}$ that are bounded by depth d . For simplicity, we use $C \in \mathbb{C}$.

Zero-Knowledge Proof with Preprocessing (PZK)

Here, a specialized type of zero-knowledge proof protocol is being considered where nearly all communications (except the last) are offline [11]. Pre-processing yields σ_V for the verifier and σ_P for the prover. The online phase requires one round, a relaxed, non-interactive zero-knowledge proof. This pre-processing zero-knowledge proof (PZK) is achievable under weaker assumptions than standard NIZK (e.g., one-way functions).

Definition 1. Let $L \subseteq \{0, 1\}^*$ be a language and $R \subseteq L \times \{0, 1\}^*$ a relation. A triple of PPT algorithms $\text{PZK} = (\text{PZK.Pre}, \text{PZK.Prove}, \text{PZK.Verify})$ is said to be a zero-knowledge proof system with preprocessing if, upon executing $(\sigma_V, \sigma_P) \leftarrow \text{PZK.Pre}(1^\lambda)$, it holds that for all $(x, w) \in R$, $\Pr[\text{PZK.Verify}(\sigma_V, x, \pi) = 1 : \pi \leftarrow \text{PZK.Prove}(\sigma_P, x, w)] = 1$, while for every $x \notin L$ we have $\Pr[\exists \pi : \text{PZK.Verify}(\sigma_V, x, \pi) = 1] = \text{negl}(\lambda)$. Moreover,

there exists a PPT simulator $\mathcal{S}$ such that for every $(x, w) \in R$ the ensembles $\{\sigma_V, \text{PZK.Prove}(\sigma_P, x, w)\}$ and $\{\mathcal{S}(x)\}$ are computationally indistinguishable.

Threshold Fully Homomorphic Encryption (TFHE)

A (leveled) threshold fully homomorphic encryption scheme [12] relies on efficient access structures $(\mathbb{S}, N)$. These access structures formalize the authorized participant subsets $S \in \mathbb{S}$ capable of collaborative decryption, where a valid threshold scheme must satisfy both correctness and security properties. Specifically, the correctness property requires that $\forall S \in \mathbb{S}, \forall \mu \in \{0, 1\}$, and for any circuit $C \in \mathbb{C}$ of depth at most d : $\text{FinDec}(\text{pk}, \{\text{PartDec}(\text{pk}, \text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k), \text{sk}_i)\}_{i \in S}) = C(\mu_1, \dots, \mu_k)$ where $\text{ct}_j \leftarrow \text{Encrypt}(\text{pk}, \mu_j)$ [4].

Definition 2. Consider the efficient access structures $(\mathbb{S}, N)$ and circuit class $(\mathbb{C}, d)$, a *threshold fully homomorphic encryption* scheme consists of $\text{TFHE} = (\text{Setup}, \text{Encrypt}, \text{Eval}, \text{PartDec}, \text{FinDec})$.

- $(\text{pk}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{Setup}(1^\lambda, 1^d, (\mathbb{S}, N))$ produces the public key pk and a collection of secret key shares $\text{sk}_1, \dots, \text{sk}_N$, each corresponding to one party, given the security parameter 1^λ , a bound 1^d on the circuit depth, and an efficient access structure $(\mathbb{S}, N)$.
- $\text{ct} \leftarrow \text{Encrypt}(\text{pk}, \mu)$: returns the ciphertext ct , given the public key pk and the plaintext $\mu \in \{0, 1\}$.
- $\hat{\text{ct}} \leftarrow \text{Eval}(\text{pk}, C, \text{ct}_1, \dots, \text{ct}_k)$. For any circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$ of depth at most d and ciphertexts $\text{ct}_1, \dots, \text{ct}_k$, the evaluation algorithm computes an evaluated ciphertext $\hat{\text{ct}}$.
- $\text{p}_i \leftarrow \text{PartDec}(\text{pk}, \text{ct}, \text{sk}_i)$. Given the secret share sk_i and for any ciphertext ct , the party P_i computes a decryption share p_i .
- $\hat{\mu} \leftarrow \text{FinDec}(\text{pk}, \{\text{p}_i\}_{i \in S})$. Considering a set $S \subset \mathbb{S}$ and the associated decryption shares $\{\text{p}_i\}_{i \in S}$, the final decryption algorithm reconstructs the plaintext $\hat{\mu} \in \{0, 1, \perp\}$.

Perfectly Binding Non-interactive Commitment (Com)

The work of Blum [13] builds non-interactive commitments from injective one-way functions, and [14] builds them from regular one-way functions and a worst-case assumption relating deterministic time $2^{O(n)}$ and non-deterministic circuit complexity $2^{\Omega(n)}$. Recent work [15] constructs such schemes from LWE.

Definition 3. A non-interactive commitment scheme is an algorithm $C = (\text{Com})$ satisfying the following conditions. For message $x \in \{0, 1\}^*$ and randomness $r \in \{0, 1\}^\lambda$, let $\text{com} \leftarrow C.\text{Com}(x; r)$; the scheme must fulfill Perfect Binding and Computational Hiding requirements respectively as specified below:

- For every security parameter $\lambda \in \mathbb{N}$, and string $\text{com} \in \{0, 1\}^*$, there exists at most a single $x \in \{0, 1\}^*$ such that com is a commitment to x : Let the binding property be for any $r_0, r_1 \in \{0, 1\}^\lambda$:

$$C.\text{Com}(x_0; r_0) = C.\text{Com}(x_1; r_1) \implies x_0 = x_1.$$

- For every $\lambda \in \mathbb{N}, x_0, x_1 \in \{0, 1\}^{\text{poly}(\lambda)}$, the following distributions are computationally indistinguishable: Let $\mathcal{D}_i = \{\text{com}_i : \text{com}_i \stackrel{R}{\leftarrow} C.\text{Com}(x_i; r)\}_{r \leftarrow \{0, 1\}^\lambda}$ for $i \in \{0, 1\}$, then, $\mathcal{D}_0 \approx_c \mathcal{D}_1$.

III. INTUITION ON UNIVERSAL THRESHOLDIZER

The concept of TFHE naturally extends conventional FHE by incorporating threshold mechanisms, which makes it a versatile building block for threshold cryptographic applications. In many practical scenarios, rather than employing the full generality of TFHE, one may only need its capability to thresholdize any basic cryptographic function [16]. In such contexts, TFHE is better viewed as a thresholdizer tool. For its effective use in this role, it is essential that the scheme is robust—that is, there must be an efficient public procedure available to verify that a given partial decryption is correct. This leads us to the notion of the universal thresholdizer (UT), which encapsulates these desirable properties.

Generally, a universal thresholdizer merges the TFHE setup and encryption stages into a single UT.Setup algorithm, while combining the evaluation and partial decryption operations into a unified UT.Eval algorithm. For simplicity, the separate definitions of semantic security and simulation security for TFHE are also combined into one overarching security requirement. Additionally, the UT model introduces a dedicated verification procedure to ensure that the computed evaluations are correct.

Definition 4 (Static Universal Thresholdizer). Consider the efficient access structure $(\mathbb{S}, N)$ and circuit class $(\mathbb{C}, d)$. A *static universal thresholdizer* UT consists of the algorithms:

- $(\text{p}, s_1, \dots, s_N) \leftarrow \text{Setup}(1^\lambda, 1^d, (\mathbb{S}, N), x)$: Given the security parameter 1^λ , a depth bound 1^d , an efficient access structure $(\mathbb{S}, N)$, and a message $x \in \{0, 1\}^k$, this algorithm outputs public parameters pp and a set of shares $s_1, \dots, s_N$.
- $y_i \leftarrow \text{Eval}(\text{p}, s_i, C)$: On input of the public parameters p , a share s_i , and a circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$ of depth at most d , the algorithm produces a partial evaluation y_i .
- $\{0, 1\} \leftarrow \text{Verify}(\text{p}, y_i, C)$: This algorithm takes as input p , a purported partial evaluation y_i , and a circuit C , and outputs a bit indicating whether the evaluation is valid.
- $y \leftarrow \text{Combine}(\text{p}, B)$: This function considers $B = \{y_i\}_{i \in S}$ is a collection of partial evaluations from an authorized set S . The algorithm outputs the final evaluation y .

We state that UT (Definition 4) has a static structure; there is a tiny gap in achieving UT and in order to present the evolution of UT 's construction methods in a complete manner, we introduce the concept of Dynamic Universal Thresholdizer (dUT), which differs from Static Universal Thresholdizer (UT).

Definition 5 (Dynamic Universal Thresholdizer). Given the efficient access structure $(\mathbb{S}, n)$ and circuit class $(\mathbb{C}, d)$, a

$\text{Exp}_{\mathcal{A}, \text{Real}}^{\text{dUTsec}}(1^\lambda, 1^d)$	$\text{Exp}_{\mathcal{A}, \text{Ideal}}^{\text{dUTsec}}(1^\lambda, 1^d)$
1: $b \leftarrow 0$	1: $b \leftarrow 1$
2: $((\mathbb{S}, N), (C, d)) \leftarrow \mathcal{A}(1^\lambda, 1^d)$	2: $((\mathbb{S}, N), (C, d)) \leftarrow \mathcal{A}(1^\lambda, 1^d)$
3: $(\text{upp}, \text{usk}_1, \dots, \text{usk}_N) \leftarrow \text{Setup}(1^\lambda, (\mathbb{S}, N), (C, d))$	3: $(\text{upp}, \text{usk}_1, \dots, \text{usk}_N, \text{st}) \leftarrow \text{Sim}_1(1^\lambda, (\mathbb{S}, N), (C, d))$
4: $S^* \leftarrow \mathcal{A}(\text{upp})$	4: $S^* \leftarrow \mathcal{A}(\text{upp})$
5: $(x, C) \leftarrow \mathcal{A}(\{\text{usk}_i\}_{i \in S^*})$	5: $(x, C) \leftarrow \mathcal{A}(\{\text{usk}_i\}_{i \in S^*})$
6: $\text{ct} \leftarrow \text{Encode}(\text{upp}, x)$	6: $\text{ct} \leftarrow \text{Encode}(\text{upp}, x)$
7: $b' \leftarrow \mathcal{A}^O(\text{ct})$	7: $b' \leftarrow \mathcal{A}^O(\text{ct})$
8: return $(b = b' \wedge S^* \notin \mathbb{S})$	8: return $(b = b' \wedge S^* \notin \mathbb{S})$
$O(\mathbb{S}, C')$ / oracle $\forall i \in S. y_i \leftarrow \text{Eval}(\text{upp}, \text{usk}_i, (C', d), \text{ct})$ return $(\{y_i\}_{i \in S})$	

Fig. 1: Security Experiments for Dynamic Universal Thresholdizer

dynamic universal thresholdizer dUT consists of the following algorithms:

- $(\text{upp}, \text{usk}_1, \dots, \text{usk}_n) \leftarrow \text{Setup}(\lambda, (\mathbb{S}, n), (C, d))$. Returns the public parameters upp and for each party i they receive an individual secret key usk_i .
- $\text{ct} \leftarrow \text{Encode}(\text{upp}, x)$. Encodes the private value x into ct .
- $y_i \leftarrow \text{Eval}(\text{upp}, \text{usk}_i, (C, d), \text{ct})$. Produces the partial evaluation y_i over the encoding ct using circuit C and the secret key user i .
- $\{0, 1\} \leftarrow \text{Verify}(\text{upp}, y_i, (C, d), \text{ct})$. Verifies if the partial evaluation y_i is valid for circuit (C, d) and encoding ct . *Evaluation correctness* ensures with overwhelming probability that $1 \leftarrow \text{Verify}(\text{upp}, \text{Eval}(\text{upp}, \text{usk}_i, (C, d), \text{ct}), (C, d), \text{ct})$.
- $y \leftarrow \text{Combine}(\text{upp}, \{y_i\}_{i \in S}, \text{ct})$. Returns the combine evaluation y from the partial evaluations $\{y_i\}_{i \in S}$. *Combine correctness* ensures with overwhelming probability that $y = C(x)$ for any $S \subset \mathbb{S}$ and $y_i \leftarrow \text{Eval}(\text{upp}, \text{usk}_i, (C, d), \text{ct})$ and $\text{ct} \leftarrow \text{Encode}(\text{upp}, x)$.

In dUT, parties can evaluate different values without the need to commit to x . Boneh et al. [4] proposed a static universal thresholdizer where the value x is hard-coded in a privacy-preserving way in the public parameters - this is equivalent to calling $\text{ct} \leftarrow \text{Encode}$ in Setup and having ct contained in the public parameters upp .

Definition 6. dUT is *secure* if for any efficient adversary $\mathcal{A}$, any security parameter λ , circuit class (C, d) , access structure $(\mathbb{S}, N)$ there exists an efficient simulator $\text{Sim} = (\text{Sim}_1, \text{Sim}_2)$ with the following advantage negligible: Let $P_{\text{Real}} = \Pr[\text{Exp}_{\mathcal{A}, \text{Real}}^{\text{dUTsec}}(1^\lambda, 1^d) = 1]$ and $P_{\text{Ideal}} = \Pr[\text{Exp}_{\mathcal{A}, \text{Ideal}}^{\text{dUTsec}}(1^\lambda, 1^d) = 1]$, then $\text{Adv}_{\mathcal{A}, \text{TFHE}}^{\text{sec}}(\lambda) = |P_{\text{Real}} - P_{\text{Ideal}}|$, where $\text{Exp}_{\mathcal{A}, b}^{\text{dUTsec}}$ defined in the Figure 1.

Definition 7. A Dynamic Universal Thresholdizer scheme satisfies *robustness* if for any efficient adversary $\mathcal{A}$, any security parameter λ , circuit class (C, d) , and access structure

$(\mathbb{S}, N)$, the following advantage is negligible:

$$\Pr \left[\begin{array}{l} (\mathbb{S}, N) \leftarrow \mathcal{A}(1^\lambda, 1^d); \\ (pp, sk_1, \dots, sk_N) \leftarrow \text{Setup}(1^\lambda, (\mathbb{S}, N), (C, d)); \\ (x, C^*, i^*) \leftarrow \mathcal{A}(pp, sk_1, \dots, sk_N); \\ \text{ct} \leftarrow \text{Encode}(pp, x); \\ y_{i^*} = (p_{i^*}, \pi_{i^*}) \leftarrow \text{Eval}(pp, sk_{i^*}, (C^*, d), \text{ct}); \\ y^* = (p^*, \pi^*) \leftarrow \mathcal{A}(\text{ct}); \\ b_1 \leftarrow (p^* \neq p_{i^*}); \\ b_2 \leftarrow \text{Verify}(pp, y^*, (C^*, d), \text{ct}); \\ \text{return } (b_1 \wedge b_2) \end{array} \right]$$

The relation between Static and Dynamic Universal Thresholdizers is stated in the following lemma.

Lemma 1. Let $\text{UT} = (\text{Setup}, \text{Eval}, \text{Verify}, \text{Combine})$ be a secure Static Universal Thresholdizer and $\text{dUT} = (\text{Setup}, \text{Encode}, \text{Eval}, \text{Verify}, \text{Combine})$ be a secure Dynamic Universal Thresholdizer. Then:

- (i) Any dUT can be used to construct a UT as follows:
 $\text{UT.Setup}(1^\lambda, 1^d, (\mathbb{S}, N), x) :=$
 $(pp', sk_1, \dots, sk_N) \leftarrow \text{dUT.Setup}(1^\lambda, (\mathbb{S}, N), (C, d))$
 $\text{ct} \leftarrow \text{dUT.Encode}(pp', x)$
return $(pp = (pp', \text{ct}), sk_1, \dots, sk_N)$

(ii) UT does not generally imply dUT without additional assumptions, as UT inherently binds the secret value x during setup, precluding the dynamic encoding of arbitrary values post-setup. Therefore, dUT is more expressive than UT.

As a result, as it is tangible, the property arguments, including statements of security and robustness, for the dUT and UT are straightforward modifications of each other, echoing the arguments in [4] as they are 1:1 parallel to the UT setting.

It is worth noting that by focusing on fake partial evaluations in security and robustness, as part of formalizing security, there is an assumption that an efficient simulator exists that can produce good fake partial evaluations that no efficient adversary can distinguish from real partial evaluations. This should also hold in the context of the adversary verifying the fake partial evaluations - in essence, the verification should

return true on a fake partial evaluation that was produced by the simulator. We want to clarify that this does not contradict robustness, where the adversary itself has to produce those fake partial evaluations. The core difference comes from the fact that the simulator manipulates the setup phase to introduce a trapdoor that allows it to do this - this is exactly the same strategy that most zero-knowledge proofs take. While the adversary in the robustness is given the outcome of the setup.

IV. DESIGNING TECHNIQUES FOR UNIVERSAL THRESHOLDIZER

In this section, we study the methods of constructing UT from different aspects, and Figure 2 presents the taxonomy of building techniques, focusing on designing procedures and constructing methods of UT.

The aim of UT is to compute $C(x)$ from a private value $x \in \{0, 1\}^k$ for some public circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$. This is achieved by involving n parties that produce partial evaluations y_i with $i \in S$ and $S \subset \mathbb{S}$, and a mechanism to combine these partial evaluations to produce the final evaluation $y \leftarrow C(x)$.

UT relies on limiting an efficient adversary's ability to distinguish between a real partial evaluation and a simulated one (for *security*) and to produce fake partial evaluations that verify (for *robustness*). Additionally, UT satisfies *compactness* - the size of the partial evaluations are reasonable (i.e., $|y_i| \leq \text{poly}(\lambda, d, n)$), and *correctness* - an honest execution produces the intended outcomes.

A. Dynamic Universal Thresholdizer

To construct a dUT, as Figure 3 shows, utilizing TFHE is enough. It is straightforward to see that if one designs UT only from TFHE it produces an UT that is compact, correct, and secure but not robust. Consider $\text{ct} \leftarrow \text{TFHE.Enc}(\text{tpk}, x)$, $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tpk}, C, \text{ct})$ and $y_i \leftarrow \text{TFHE.PartDec}(\text{tsk}_i, \hat{\text{ct}})$. The UT security and TFHE simulated security are identical, and both ask the adversary to distinguish between real and simulated values for y_i . To obtain robustness for UT, a method to verify the partial evaluation is needed; however, TFHE does not provide one, and as such, we can only have $1 \leftarrow \text{Verify}(\text{upp}, y_i, (C, d), \text{ct})$. This naturally implies that any value $z \leftarrow \mathbb{Z}_q$ will also successfully verify. One can obtain a trivial static equivalent for UT by creating ct in setup and adding it to the public parameters upp .

B. Boneh et al. [4], $\text{UT} = \text{TFHE} + \text{PZK} + \text{Com}$

The first construction of UT is about generating multiple key shares from a given cryptographic key. Each share allows for a partial evaluation of a function, and these partial results are then combined to yield the function's final output. This method utilizes TFHE, which is based on $\{0, 1\}$ -LSSS and FHE, along with PZK [11] and a commitment scheme C to achieve universal thresholdizer as Figure 4 shows. Apart from $\text{TFHE} + \text{PZK} + C$, Boneh et al. propose another way of constructing UT with $\text{TFHE} + \text{HS}$ as a more efficient solution

from the robustness dimension, but they do not construct and formalize it.

This static UT has been proposed by Boneh et al [4] and has been shown to be compact, correct, secure and robust. Consider $\text{ct} \leftarrow \text{TFHE.Enc}(\text{tpk}, x)$, $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tpk}, C, \text{ct})$. The idea is to have the partial evaluation $y_i = (p_i, \pi_i)$ composed from TFHE partial decryption $p_i \leftarrow \text{TFHE.PartDec}(\text{tsk}_i, \hat{\text{ct}})$ and prove using π_i that p_i is obtained from decrypting $\hat{\text{ct}}$ and that the right key was used (this is where the commitment to tsk_i in setup is needed). The security of UT trivially reduces to the simulated security of TFHE, the zero-knowledge property of the proof system PZK, and the computational-binding property of the commitment scheme Com . Robustness only holds if the experiment actually measures improperly computed partial evaluations.

The technique of Figure 4 provides a solid balance of time and space efficiency with linear N scaling. However, from a design dimension, using HS is a better option than PZK to ensure robustness. Because homomorphic signatures [27] give more compact proofs than PZKs, we can get partial evaluations y_i whose size is independent of the original secret message x and the size of circuit C that is used for the evaluation [4]. Thus, HS are more efficient than PZK+Com in building UT from the robustness aspect.

In addition, there exists another limitation on the general idea of universal thresholdizer's construction, which is utilizing PZK. NIZKs have $\Theta(1)$ round complexity (one prover message after CRS). PZKs need $\Omega(k)$ preprocessing rounds plus proof rounds, thus asymptotically less efficient under standard assumptions [28]. NIZKs offer better universal composability, maintaining simulation-extractability concurrently without rewinding (which breaks parallel PZKs security), and stronger non-malleability via existentially unforgeable proofs resisting attacks on (x, π) [29].

Moreover, PZKs demonstrate susceptibility to rewinding attacks where malicious verifiers can extract witnesses by exploiting the interactive preprocessing phase, violating the knowledge soundness property when the verifier deviates from the prescribed protocol [30], [31]. Moreover, PZKs frequently suffer from selective failure vulnerabilities whereby a malicious verifier can learn predicate information about the witness by observing aborts in specific preprocessing configurations, effectively reducing the zero-knowledge guarantee to witness-indistinguishability rather than maintaining full simulation-based security [32], [33]. Additionally, PZKs typically require stronger trust assumptions regarding the preprocessing phase setup, creating potentially exploitable single points of failure that NIZKs with distributed CRS generation protocols can mitigate through threshold security models [34].

However, NIZK can not be used in the intended construction because UT is based on lattice structures, and obtaining NIZK from lattices is still an open problem [35]. That is why PZK has been used in the universal thresholdizer [4]; i.e., their setting allows the use of NIZK with preprocessing, which can be constructed from one-way functions and, therefore, can be instantiated from lattices.

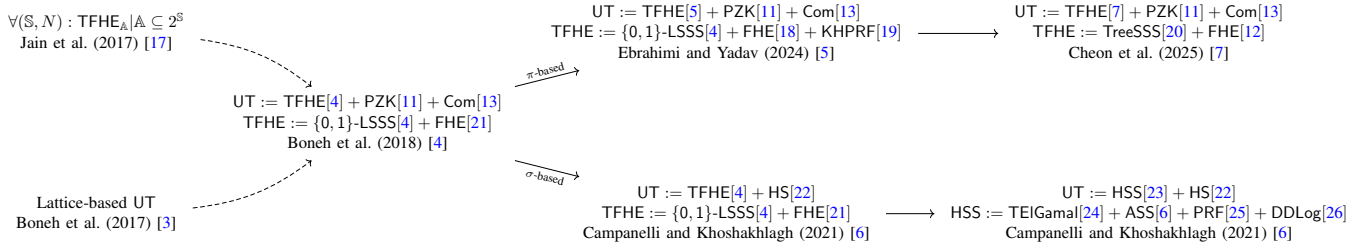


Fig. 2: Construction Methods of Universal Thresholdizer

dUT.Setup ($1^\lambda, (\mathbb{S}, N), (C, d)$)	dUT.Eval ($\text{upp}, \text{usk}_i, (C, d), \text{ct}$)
1 : $(\text{tfhepk}, \text{tfhesk}_1, \dots, \text{tfhesk}_N) \leftarrow \text{TFHE.Setup}(1^\lambda, 1^d, \mathbb{S})$	1 : $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{upp}, C, \text{ct})$
2 : $\text{upp} \leftarrow \text{tfhepk}$	2 : $y_i \leftarrow \text{TFHE.PartDec}(\text{upp}, \hat{\text{ct}}, \text{usk}_i)$
3 : $\forall i \in [N], \text{usk}_i \leftarrow \text{tfhesk}_i$	3 : return y_i
4 : return $(\text{upp}, \text{usk}_1, \dots, \text{usk}_N)$	
dUT.Encode (upp, x)	dUT.Verify ($\text{upp}, y_i, (C, d), \text{ct}$)
1 : $\forall i \in [k], \text{ct}_i \leftarrow \text{TFHE.Encrypt}(\text{upp}, x_i)$	1 : return 1 / TFHE provides no way to verify
2 : $\text{ct} \leftarrow \{\text{ct}_i\}_{i \in [k]}$	
3 : return ct	dUT.Combine ($\text{upp}, \{y_i\}_{i \in S}, \text{ct}$)
	1 : $S \subseteq \{1, \dots, N\} \mid S \in \mathbb{S}$
	2 : $y \leftarrow \text{TFHE.FinDec}(\text{upp}, \{y_i\}_{i \in S})$
	3 : return y

Fig. 3: Dynamic Universal Thresholdizer from TFHE

UT.Setup ($1^\lambda, 1^d, \mathbb{A}, x$)	UT.Eval (pp, s_i, C)
1 : $(\text{tfhepk}, \text{tfhesk}_1, \dots, \text{tfhesk}_N) \leftarrow \text{TFHE.Setup}(1^\lambda, 1^d, \mathbb{A})$	1 : $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tfhepk}, C, \{\text{ct}_i\}_{i=1}^k)$
2 : $\forall i \in [k], \text{ct}_i \leftarrow \text{TFHE.Encrypt}(\text{tfhepk}, x_i)$	2 : $p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$
3 : $\forall i \in [N], (\sigma_{V,i}, \sigma_{P,i}) \leftarrow \text{PZK.Pre}(1^\lambda)$	3 : $\Psi_i : \exists (\text{tfhesk}_i, r_i) \mid \text{com}_i \leftarrow \text{C.Com}(\text{tfhesk}_i; r_i)$ $\wedge p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$
4 : $\forall i \in [N], r_i \xleftarrow{R} \{0, 1\}^\lambda$	4 : $\pi_i \leftarrow \text{PZK.Prove}(\sigma_{P,i}, \Psi_i, (\text{tfhesk}_i, r_i))$
5 : $\forall i \in [N], \text{com}_i \leftarrow \text{C.Com}(\text{tfhesk}_i; r_i)$	5 : $y_i \leftarrow (p_i, \pi_i)$
6 : $\text{pp} \leftarrow (\text{tfhepk}, \{\text{ct}_i\}_{i \in [k]}, \{\sigma_{V,i}\}_{i \in [N]}, \{\text{com}_i\}_{i \in [N]})$	6 : return y_i
7 : $\forall i \in [N], s_i \leftarrow (\text{tfhesk}_i, \sigma_{P,i}, r_i)$	
8 : return $(\text{pp}, s_1, \dots, s_N)$	UT.Verify (pp, y_i, C)
UT.Combine (pp, B)	1 : $\hat{\text{ct}} \leftarrow \text{TFHE.Eval}(\text{tfhepk}, C, \{\text{ct}_i\}_{i=1}^k)$
1 : $B \leftarrow \{y_i\}_{i \in S} \mid (S \subseteq \{1, \dots, N\}) \vee (S \in \mathbb{A})$	2 : $\Psi_i : \exists (\text{tfhesk}_i, r_i) \mid \text{com}_i \leftarrow \text{C.Com}(\text{tfhesk}_i; r_i)$ $\wedge p_i \leftarrow \text{TFHE.PartDec}(\text{tfhepk}, \hat{\text{ct}}, \text{tfhesk}_i)$
2 : $\forall i \in S : y_i \leftarrow (p_i, \pi_i)$	3 : $y_i \leftarrow (p_i, \pi_i)$
3 : $y \leftarrow \text{TFHE.FinDec}(\text{tfhepk}, \{p_i\}_{i \in S})$	4 : return $\text{PZK.Verify}(\sigma_{V,i}, \Psi_i, \pi_i)$
4 : return y	

Fig. 4: Universal Thresholdizer from TFHE+PZK+C, Boneh et al. (2018) [4]

In addition, Boneh et al. (2018) [4] proposed two separate TFHE schemes (one based on Shamir and one based on $0,1$ -LSSS), and recently, an idea of attack on Shamir-based t -out-of- N TFHE has been proposed [36], which breaks the simulation security, and it seems insecure due to the adversary's ability to recover secret key under constraints on t and N .

C. Campanelli and Khoshakhlagh (2021) [6],
 $\text{UT} = \text{TFHE} + \text{HS}$

To address the aforementioned limitation, [6] presented constructions based on HS. Homomorphic signatures [27] give more compact proofs than NIZKs, and we can get partial evaluations y_i whose size is independent of the original secret message x and the size of the circuit C that is used for the evaluation. Unlike NIZK, homomorphic signatures can be constructed from the SIS problem [37]. The first procedure of [6] is constructing UT using TFHE+HS, as demonstrated in

UT.Setup ($1^\lambda, d, N, x$) 1 : $(\text{skhs}, \text{pkhs}) \leftarrow \text{HS.Setup}(1^\lambda)$ 2 : $(\text{skfhe}, \text{pkfhe}) \leftarrow \text{TFHE.KeyGenAux}(1^\lambda, d, N)$ 3 : $\text{ctx} \leftarrow \text{TFHE.Enc}(\text{pkfhe}, x)$ 4 : $\text{sk} \leftarrow \text{UT} - \text{AuxSetup}(\text{skfhe}, \text{skhs})$ 5 : return $(\text{pp} := (\text{pkfhe}, \text{pkhs}, \text{ctx}), \text{sk})$	UT.PartEval ($\text{pp} := (\text{pkfhe}, \text{pkhs}, \text{ctx}), \text{ski}, i, C$) 1 : $\text{ct}' \leftarrow \text{TFHE.Eval}(\text{pkfhe}, \text{ctx}, C)$ 2 : $y_i \leftarrow \text{TFHE.PartDec}(\text{ski}[\text{fhe}], \text{ct}')$ 3 : $\sigma_i \leftarrow \text{HS.Eval}(\text{pkhs}, "i", C_{\text{Dec}}, \text{ski}[\text{hs}])$ $C_{\text{Dec}} := \text{TFHE.PartDec}(\cdot, \text{ct}')$ 4 : return (y_i, σ_i)
UT-AuxSetup ($\text{skfhe}, \text{skhs}$) 1 : $\forall i \in \{1, \dots, N\}$ 2 : $\text{sk}'_i[\text{fhe}] \leftarrow \text{SS}(d, N, \text{skfhe})$ 3 : $\text{sk}'_i[\text{hs}] \leftarrow \text{HS.Sign}(\text{skhs}, "i", \text{sk}'_i[\text{fhe}])$ 4 : $\text{sk}'_i := (\text{sk}'_i[\text{fhe}], \text{sk}'_i[\text{hs}])$ 5 : return $\text{sk}'_1, \dots, \text{sk}'_N$	UT.VfyEval ($\text{pp} := (\text{pkfhe}, \text{pkhs}), \pi_i = (y_i, \sigma_i), i, C$) 1 : $\text{ct}' \leftarrow \text{TFHE.Eval}(\text{pkfhe}, \text{ctx}, C)$ 2 : return $\text{HS.Verify}(\text{pkhs}, "i", y_i, \sigma_i, C_{\text{Dec}})$ $C_{\text{Dec}} := \text{TFHE.PartDec}(\cdot, \text{ct}')$
	UT.Combine ($\text{pp} := (\text{pkfhe}, \text{pkhs}), y_1, \dots, y_d$) 1 : return $\text{TFHE.Dec}(\text{pkfhe}, \{y_1, \dots, y_d\})$

Fig. 5: Universal Thresholdizer from TFHE+HS, Campanelli and Khoshakhlagh (2021) [6]

the Figure 5.

Boneh et al. [4] provided the initial idea of TFHE+HS but the formal definition and construction presented in Campanelli and Khoshakhlagh (2021) [6].

The advantage of this technique is that the size of certificates (output of UT.PartEval) for checking each party's cooperation is more efficient than TFHE+PZK+C because the size of the homomorphic signatures depends on the depth of the computation being certified, which in this case scales logarithmically with the statement size.

Campanelli and Khoshakhlagh [6] claim this is compact, correct, secure, and robust. We will disprove that claim in Section V and show that we can have either robustness or security, but not both at the same time, for this construction. We also want to clarify that no formal proof has been given to $\text{UT} = \text{TFHE} + \text{HS}$ by [6]. They define robustness but do not provide explicit proof for the general construction, and they lack the guarantee that adversaries cannot forge valid certificates. Notice that Boneh et al [4] stated that for robustness, one can use HS instead of PZK + Com, and do not say (or imply) that HS is sufficient for security.

Another important error made by [6] is providing an incorrect definition for the *context-hiding* property for homomorphic signatures. They claim it models indistinguishability between a simulated signature (from a simulator with access to the HS secret key) and a signature that was updated over a circuit. This is incorrect as the definition laid out by [38] states this property is for distinguishing between a direct signature on $C(m)$ and homomorphically building this from a signature over m . Additionally, it is a well-known convention that simulators do not have access to secret keys - they are supposed to "simulate" the protocol, not run it.

D. Campanelli and Khoshakhlagh (2021) [6],
 $\text{UT} = \text{HSS} + \text{HS}$

The other technique presented in [6] is a (2, 2)-case for UT, which is subsumed by Construction ?? (Subsumed Function-

ality). This type of UT is getting built upon HSS and HS as shown in Figure 6.

This HSS-based construction requires simpler primitives and is more efficient than TFHE [39], and allows for a wider type of instantiations, for example, from DDH as in [23].

Despite the Construction ?? that works for any threshold access structure (d, N) , this one only works for exactly two parties with a threshold of 2, i.e., a (2, 2)-threshold scheme, (restricted setting) and cannot be extended to larger committees (limited scalability). In addition, this instantiation is still plausibly weaker than publicly verifiable NIZKs as explicitly mentioned in [6].

E. Ebrahimi et al. (2024) [5], $\text{UT} = \text{TFHE} + \text{PZK} + \text{Com}$

The previous techniques assumed the standard security of UT, but in [5], they not only attempt to construct UT efficiently but also provide a stronger definition of simulation security for UT. This method improves the TFHE construction from [4] using key-homomorphic PRF techniques, and strengthens security definitions inspired by Bellare's works [40] and [41]. Additionally, they combine their techniques with Agrawal et al. (2022) [42] to achieve partially adaptive key queries. Unlike other ideas, this procedure achieves the thresholdizer by mixing $\{0, 1\}$ -LSSS with FHE and KHPRF.

This technique works the same as Figure 4, but the main difference lies in the way of building the new TFHE as Figure 7; i.e., the difference is in the fact that the [5] satisfies a stronger notion of simulation security, and this leads to a universal thresholdizer with a stronger notion of security. The proof is exactly the same as for [4], with only accounting for the updates in the security definitions.

In some real-world applications, handling lattice-based "Almost" Key-Homomorphic PRFs can be challenging in the case of UT, as they must add flooding noise to hide the error, which increases noise parameters. There is a solution to this drawback in [42], which involves using Rényi divergence-based analysis [43] to optimize noise parameters in threshold signature schemes. This solution allows reducing the size of

$UT_{(2,2)}.Setup(1^\lambda, d=2, N=2, x)$ 1 : $(skhs, pkhs) \leftarrow HS.Setup(1^\lambda)$ 2 : $sk \leftarrow UT_{(2,2)}-AuxSetup(x, skhs)$ 3 : return $(pp := pkhs, sk)$	$UT_{(2,2)}.PartEval(pp := pkhs, sk_i, i, C)$ 1 : $y_i \leftarrow HSS.Eval_i(share_i, C)$ 2 : $\sigma_i \leftarrow HS.Eval(pkhs, "i", C_{HSSEval}, sk_i[hs])$ $ C_{HSSEval} := HSS.Eval_i(\cdot, C)$ 3 : return (y_i, σ_i)
$UT_{(2,2)}-AuxSetup(x, skhs)$ 1 : $(share_1, share_2) \leftarrow HSS.Share(1^\lambda, x)$ 2 : $\forall i \in \{1, 2\}$ 3 : $sk'_i[hs] \leftarrow HS.Sign(skhs, "i", share_i)$ 4 : $sk'_i := (share_i, sk'_i[hs])$ 5 : return (sk'_1, sk'_2)	$UT_{(2,2)}.VfyEval(pp := pkhs, \pi_i = (y_i, \sigma_i), i, C)$ 1 : return $HS.Verify(pkhs, "i", y_i, \sigma_i, C_{HSSEval})$ $ C_{HSSEval} := HSS.Eval_i(\cdot, C)$
	$UT_{(2,2)}.Combine(pp := pkhs, y_1, y_2)$ 1 : return $HSS.Combine(y_1, y_2)$

Fig. 6: Two-Party Universal Thresholdizer from HSS+HS, Campanelli and Khoshakhlagh (2021) [6]

$TFHE.Setup(1^\lambda, 1^d, A_{t,n})$ 1 : $(fpk, fsk) \leftarrow FHE.Setup(1^\lambda, 1^d)$ 2 : $(K_1, K_2, \dots, K_n) \leftarrow bSS.Share(0 \in \mathcal{K}, A_{t,n})$ 3 : $(fsk_1, fsk_2, \dots, fsk_n) \leftarrow bSS.Share(fsk, A_{t,n})$ 4 : $\forall i \in [n]. tfsk_i := (\{fhesk_j\}_{j \in T_i}, \{k_j\}_{j \in T_i})$ 5 : return $(tfpk := fpk, \{tfsk_i\}_{i \in [n]})$	$TFHE.PartDec(tfpk, tfsk_i, ct)$ 1 : $(\{fhesk_j\}_{j \in T_i}, \{k_j\}_{j \in T_i}) := tfsk_i$ 2 : $\forall j \in T_i. \xi_j \xleftarrow{R} [-B_{sm}, B_{sm}]$ 3 : $\forall j \in T_i. \eta_j \xleftarrow{R} [-E_{sm}, E_{sm}]$ 4 : $\forall j \in T_i. y_j := FHE.decode_0(fhesk_j, ct) + \xi_j + F(k_j, H(ct)) + \eta_j$ 5 : $p_i := \{y_j\}_{j \in T_i}$ 6 : return p_i
$TFHE.Encrypt(tfpk, \mu)$ 1 : $ct \leftarrow FHE.Encrypt(tfpk, \mu)$ 2 : return ct	$TFHE.FinDec(tfpk, S, \{p_i\}_{i \in S})$ 1 : $S \subseteq [n] \mid S \in A_{t,n}$ 2 : $\forall i \in S. \{y_j\}_{j \in T_i} := p_i$ 3 : $T := \text{MinimalValidShareSet}(\cup_{i \in S} T_i)$ 4 : $\mu := FHE.decode_1(\sum_{j \in T} y_j)$ 5 : return μ
$TFHE.Eval(tfpk, C, \{ct_i\}_{i \in [k]})$ 1 : $ct_C \leftarrow FHE.Eval(tfpk, C, ct_1, \dots, ct_k)$ 2 : return ct_C	

Fig. 7: Threshold Fully Homomorphic Encryption from $\{0, 1\}$ -LSSS with FHE and KHPRF, Ebrahimi and Yadav (2024) [5].

the q from exponential to polynomial in λ . However, Ebrahimi and Yadav [5] acknowledge that this optimization cannot be applied to their TFHE and UT constructions because the Rényi divergence procedure [43] works specifically for search-based primitives and cannot be transferred to indistinguishability-based constructions.

F. Cheon et al. (2025) [7], $UT = TFHE + PZK + Com$

After Ebrahimi et al. (2024) [5], a communication-efficient one-round TFHE construction based on a specialized linear secret sharing scheme, called TreeSSS and constructed through the iterative application of s -out-of- $(2s - 1)$ Shamir secret sharing, where $s \ll N$, presented in [7]. This construction reduces the number of required secret shares from $O(N^{5.3})$ in previous $\{0, 1\}$ -LSSS schemes to $O(N^{3+o(1)})$ while maintaining the compactness property of UT. The result matches the optimal amplification used in constructing large input majority functions from smaller ones. TreeSSS can also serve as a subroutine for constructing lattice-based t -out-of- N threshold primitives such as TFHE and threshold signatures, individually. This method is presented in Figure 8.

This technique inherits a similar limitation to Boneh's $\{0, 1\}$ -LSSS construction: the need for exponential offline verification costs. Both approaches require checking whether a share matrix M is correctly generated, which involves $O(\binom{N}{t})$ operations, resulting in exponential time complexity for large N , so both methods share certain fundamental limitations related to probabilistic construction and offline verification costs.

V. BALANCING SIMULATION SECURITY AND ROBUSTNESS

Here, we will go through the universal thresholdizer's robustness and security issues that we found during our study, along with their solutions.

A. Incomplete UT Robustness

Initially, the robustness for UT is defined by Boneh et al [4] (Definition 10). In the robustness experiment by Boneh et al, the circuit C used to verify the adversarial partial evaluation y_i^* is left undefined. It seems the intuition is for the adversary to also produce this circuit when it produces the partial evaluation, as it controls all user secret keys.

TFHE.Setup ($1^\lambda, 1^d, \mathcal{A}_{N,t}$) 1: $(\text{fhepk}, \text{fhesk}) \leftarrow \text{FHE.Setup}(1^\lambda, 1^d)$ 2: $(\text{share}_1^{(L)}, \dots, \text{share}_{(2s-1)L}^{(L)}) \leftarrow \text{TreeSSS.Share}(\text{fhesk}, \mathcal{A}_{N,t})$ 3: $\forall i \in [N]. T_i := \{j \mid P_i \text{ has } \text{share}_j^{(L)}\}$ 4: $\forall i \in [N]. \text{sk}_i := \{\text{share}_j^{(L)}\}_{j \in T_i}$ 5: return $(\text{pk} := \text{fhepk}, \{\text{sk}_i\}_{i \in [N]})$	TFHE.Enc (pk, μ) 1: $\text{ct} \leftarrow \text{FHE.Enc}(\text{pk}, \mu)$ 2: return ct
TFHE.PartDec ($\text{pk}, \text{sk}_i, \hat{\text{ct}}$) 1: $\forall j \in T_i. e_j \xleftarrow{\$} [-B_{\text{sm}}, B_{\text{sm}}]$ 2: $\forall j \in T_i. \hat{p}_j^{(L)} := \text{FHE.Dec}_0(\text{share}_j^{(L)}, \hat{\text{ct}}) + ((2s-1)!)^L e_j \in \mathbb{Z}_q$ 3: $p_i := \{\hat{p}_j^{(L)}\}_{j \in T_i}$ 4: return p_i	TFHE.FinDec (pk, B) 1: if $S \notin \mathcal{A}_{N,t}$ then return $\perp$ 2: if $S \in \mathcal{A}_{N,t}$ then $T \leftarrow \min\{\mathcal{T} \mid \mathcal{T} \subseteq \bigcup_{i \in S} T_i\}$ $\hat{\mu} \leftarrow \text{FHE.Dec}_1(\sum_{j \in T} c_j^S \cdot \hat{p}_j^{(L)})$ 3: return $\hat{\mu}$

Fig. 8: Threshold Fully Homomorphic Encryption from TreeSSS and FHE, Cheon et al. (2025). [7].

Additionally, the experiment was initially formalized as the adversary input $y_i^* \neq \text{Eval}(\text{upp}, \text{usk}_i, C)$. This should be interpreted as $y_i^* \neq \{y_i \mid y_i \leftarrow \text{Eval}(\text{upp}, \text{usk}_i, C)\}$ to capture the intuition on improperly computed partial evaluation. Another approach, and the one we use in this paper, is to have split $y_i = (p_i, \pi_i)$ where there is only one true partial evaluation p_i and potentially many valid auxiliary information π_i . Thus, the adversary is restricted to produce (p_i^*, π_i^*) for $p_i^* \neq p_i$ with $(p_i, \pi_i) \leftarrow \text{Eval}(\text{upp}, \text{usk}_i, C)$.

We provide in Figure 9 the correct experiment. The robustness of dUT (Definition 7) does not inherit this incompleteness.

Exp_{A,UT,rb} ($1^\lambda, 1^d$) 1: $(x, \mathbb{A}) \leftarrow \mathcal{A}(1^\lambda, 1^d)$ 2: $(\text{pp}, \text{s}_1, \dots, \text{s}_N) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, \mathbb{A}, x)$ 3: $(y_i^*, C) \leftarrow \mathcal{A}(\text{pp}, \text{s}_1, \dots, \text{s}_N)$ 4: $(p_i, \pi_i) \leftarrow \text{UT.Eval}(\text{pp}, \text{s}_i, C)$ 5: $(p_i^*, \pi_i^*) := y_i^*$ 6: return $(\text{UT.Verify}(\text{pp}, y_i^*, C) = 1) \wedge (p_i^* \neq p_i)$
--

Fig. 9: Fixed Robustness Experiment for UT

B. Vulnerability of σ -based UT's Robustness

We provide a formal proof by presenting attacks that force a fundamental tradeoff: either security must be compromised to achieve robustness, or robustness must be sacrificed to maintain security in a signature-based universal thresholdizer.

The security-robustness impossibility of this method comes from an inherent tension in the construction: the homomorphic signature component must simultaneously bind to the computation (for robustness) while hiding information about the secret shares (for security).

Theorem 2. *Let UT be a universal thresholdizer constructed from TFHE and HS as described in Section VI-B. Then, for any*

instantiation of TFHE and HS, at least one of the following statements must be true:

- 1) UT does not satisfy security according to Definition 11.
- 2) UT does not satisfy robustness according to Definition 12.

Proof. We proceed with Theorem 2 by demonstrating that any attempt to satisfy one property necessarily violates the other. The proof proceeds by examining the requirements of security and robustness separately, then establishing their mutual incompatibility. For the UT scheme to be robust, the verification algorithm must reliably detect invalid partial evaluations. This requires the homomorphic signature scheme to enforce a binding relationship between $\text{sk}_i[\text{fhe}]$ and the resulting partial decryption y_i .

Lemma 3 (Robustness Constraint). *For UT to satisfy robustness per Definition 12, there must exist a binding relationship between $\text{sk}_i[\text{fhe}]$ and y_i that is efficiently verifiable through σ_i , specifically: For any $y_i' \neq \text{TFHE.PartDec}(\text{sk}_i[\text{fhe}], \text{ct}')$, the probability that there exists σ_i' such that $\text{HS.Verify}(\text{pk}_{\text{hs}}, "i", y_i', \sigma_i', C_{\text{Dec}}) = 1$ is negligible in λ .*

Assume by contradiction that there exists a PPT adversary $\mathcal{A}$ that can produce, with non-negligible probability, a value $y_i' \neq \text{TFHE.PartDec}(\text{sk}_i[\text{fhe}], \text{ct}')$ and a signature σ_i' such that $\text{HS.Verify}(\text{pk}_{\text{hs}}, "i", y_i', \sigma_i', C_{\text{Dec}}) = 1$. Then $\mathcal{A}$ can output $\pi_i^* = (y_i', \sigma_i')$ such that $\pi_i^* \neq \text{UT.PartEval}(p, \text{sk}_i, i, C)$ and $\text{UT.VfyEval}(p, \pi_i^*, i, C) = 1$, directly violating the robustness property. Therefore, for robustness to hold, such an adversary cannot exist, which necessitates the binding relationship described in the Lemma 3.

For the UT scheme to be secure, the partial evaluations must not leak information about the secret shares beyond what is implied by the function output.

Lemma 4 (Security Constraint). *For UT to satisfy security per Definition 11, the distribution of σ_i must be simulatable given*

only $y_i = \text{TFHE.PartDec}(\text{sk}_i[\text{fhe}], \text{ct}')$, without knowledge of $\text{sk}_i[\text{fhe}]$.

The security definition requires the existence of a simulator $\mathcal{S}_E$ that can simulate the distribution of partial evaluations without knowledge of the secret x or the shares sk_i . If σ_i depends on $\text{sk}_i[\text{fhe}]$ in a way that cannot be simulated from y_i alone, then $\mathcal{S}_E$ cannot generate a distribution indistinguishable from the real partial evaluations, violating the security requirement.

Lemma 5 (Incompatibility). *The constraints identified in Lemmas 3 and 4 cannot be simultaneously satisfied by any homomorphic signature scheme.*

We establish this incompatibility through a dilemma:

- 1) If the signature σ_i satisfies the binding requirement from Lemma 3, then it must contain or be derived from information specific to $\text{sk}_i[\text{fhe}]$ that uniquely determines y_i . This creates an information channel that leaks data about $\text{sk}_i[\text{fhe}]$.
- 2) If the signature σ_i satisfies the simulatability requirement from Lemma 4, then it cannot contain information that binds specifically to $\text{sk}_i[\text{fhe}]$, as such information would not be available to the simulator.

To formalize this incompatibility, consider an adversary $\mathcal{A}$ that controls $d - 1$ parties and targets an honest party i . The adversary has two strategies, as described below: an attack on security and/or robustness.

If binding is enforced (per Lemma 3):

- $\mathcal{A}$ obtains partial evaluations $\pi_i = (y_i, \sigma_i)$ for circuits $C_1, \dots, C_t$
- Since σ_i must bind to $\text{sk}_i[\text{fhe}]$, it contains extractable information about this share
- By analyzing multiple signatures, $\mathcal{A}$ can extract information about $\text{sk}_i[\text{fhe}]$
- With $d - 1$ corrupted shares and partial information about the d -th share, $\mathcal{A}$ can reconstruct information about sk_{fhe} , violating security

If simulatability is enforced (per Lemma 4):

- Since σ_i must be simulatable without knowledge of $\text{sk}_i[\text{fhe}]$, the binding between y_i and $\text{sk}_i[\text{fhe}]$ must be weakened
- $\mathcal{A}$ can create a fake value $y'_i \neq \text{TFHE.PartDec}(\text{sk}_i[\text{fhe}], \text{ct}')$
- Due to the weakened binding, $\mathcal{A}$ can produce a valid signature σ'_i for y'_i
- The pair (y'_i, σ'_i) passes verification, violating robustness

The two requirements are therefore in direct conflict, establishing that no homomorphic signature scheme can simultaneously satisfy both constraints.

Given Lemmas 3, 4, and 5, we can complete the proof of Theorem 2: If the UT construction prioritizes robustness, it must enforce the binding relationship required by Lemma 3, which by Lemma 5 violates the security constraint in Lemma 4. If the UT construction prioritizes security, it must enforce

the simulatability required by Lemma 4, which by Lemma 5 violates the robustness constraint in Lemma 3.

Therefore, the UT construction cannot simultaneously satisfy both security and robustness. $\square$

Attack algorithms are presented in Appendix A that demonstrate the security-robustness tradeoff established in Theorem 2. These attacks concretely illustrate how enforcing one property necessarily compromises the other in σ -based structure.

Theorem 6. *If the UT construction enforces robustness per Definition 12, then security attack (Figure 14) succeeds in distinguishing between $\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{real}}(1^\lambda, d, N)$ and $\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{ideal}}(1^\lambda, d, N)$ with non-negligible advantage.*

Proof Sketch. In the real-world experiment, the signatures σ_i^k must contain binding information about $\text{sk}_i[\text{fhe}]$ for robustness to hold (per Lemma 3). By collecting signatures for different circuits, the attacker can extract constraints on $\text{sk}_i[\text{fhe}]$. Combined with the $d - 1$ corrupted shares, these constraints enable partial reconstruction of information about sk_{fhe} . In the ideal-world experiment, the simulator $\mathcal{S}_E$ generates signatures without access to the real $\text{sk}_i[\text{fhe}]$. The distribution of these simulated signatures will differ from the real-world distribution in a way that is detectable by the algorithm's analysis, leading to a successful distinguishing attack. The advantage is non-negligible because, as established in Lemma 5, the binding required for robustness necessarily creates an information channel that cannot be perfectly simulated without knowledge of $\text{sk}_i[\text{fhe}]$. $\square$

Theorem 7. *If the UT construction enforces security per Definition 11, then robustness attack (Figure 16) succeeds in producing a fake partial evaluation $\pi_i^* = (y_i^*, \sigma_i^*)$ such that $\pi_i^* \neq \text{UT.PartEval}(p, \text{sk}_i, i, C)$ and $\text{UT.VfyEval}(p, \pi_i^*, i, C) = 1$ with non-negligible probability.*

Proof Sketch. For the UT construction to satisfy security, the signature σ_i must be simulatable without knowledge of $\text{sk}_i[\text{fhe}]$ (per Lemma 4). This simulatability requirement weakens the binding between $\text{sk}_i[\text{fhe}]$ and y_i that would be needed for robustness. The attack exploits this weakened binding to create a fake partial decryption y_i^* and a corresponding signature σ_i^* that passes verification. The success probability is non-negligible because, as established in Lemma 5, the simulatability required for security necessarily weakens the binding needed for robustness. Specifically, if the verification algorithm could reliably detect the fake partial evaluation, it would require the signature to contain binding information that cannot be simulated without knowledge of $\text{sk}_i[\text{fhe}]$, contradicting the security requirement. Therefore, when security is enforced, the attack against robustness succeeds with non-negligible probability. $\square$

One possible solution for this imbalance can be utilizing zero-knowledge proofs for HE evaluations [44] to prove the correct execution of HE evaluations while protecting the server's private inputs.

UT	Setup	Evaluation	Verification	Combination
Boneh et al. 2018 [4]	$O(N \cdot \lambda \cdot n \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(N \cdot \ell)$
Campanelli and Khoshakhlagh 2021 [6]	$O(N \cdot \ell \cdot n + n^2 \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(d \cdot n \cdot \log q)$
Campanelli and Khoshakhlagh 2021 [6]	$O(\lambda + x)$	$O(\lambda \cdot C \cdot \text{poly}(\lambda))$	$O(\lambda \cdot C \cdot \text{poly}(\lambda))$	$O(\text{poly}(\lambda))$
Ebrahimi and Yadav 2024 [5]	$O(2^d \cdot n \cdot m^2 \cdot \log q + N^{8.4})$	$O(2^d \cdot n \cdot m^2 \cdot \log q \cdot C)$	$O(2^d \cdot n \cdot m^2 \cdot \log q \cdot C)$	$O(N^{8.4})$
Cheon et al. 2025 [7]	$O(N^3 \cdot s^2 \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(C \cdot n^2 \cdot \log q)$	$O(N^2 \cdot \log q)$

TABLE I: Time Complexity of Universal Thresholdizer's Constructions

UT	Setup	Evaluation	Verification	Combination
Boneh et al. 2018 [4]	$O(N \cdot n \cdot \log q)$	$O(n \cdot \log q)$	$O(n \cdot \log q)$	$O(\ell \cdot N)$
Campanelli and Khoshakhlagh 2021 [6]	$O(N \cdot n \cdot \log q)$	$O(n \cdot \log q)$	$O(C \cdot n \cdot \log q)$	$O(d \cdot n \cdot \log q)$
Campanelli and Khoshakhlagh 2021 [6]	$O(\lambda + x)$	$O(\lambda + x)$	$O(\lambda \cdot C)$	$O(y)$
Ebrahimi and Yadav 2024 [5]	$O(N^{5.2})$	$O(n \cdot m + \Psi)$	$O(n \cdot m + \Psi)$	$O(N^{5.2})$
Cheon et al. 2025 [7]	$O(N^3 \cdot \log q)$	$O(n \cdot \log q + \Psi)$	$O(\log q + \Psi)$	$O(N^2 \cdot \log q)$

TABLE II: Space Complexity of Universal Thresholdizer's Constructions

VI. EFFICIENCY ANALYSIS OF CONSTRUCTIONS

In this section, the intention is to do an in-depth investigation of the computational costs and complexities of each technique for UT construction, where Tables I and II present the summary of our study of their efficiency.

A. Universal Thresholdizer from $\{0,1\}$ -LSSS and FHE - Boneh et al. (2018) [4]

The developers of this technique define the linear secret sharing scheme with binary reconstruction coefficients, and this class includes access structures defined by monotone Boolean formulas [45]. They then combine it with FHE [21] to achieve TFHE. SS.Share processes $k \in \mathbb{Z}_q$ in $O(1)$ time. For threshold access with N parties, formula size is $O(N^{5.2})$ [46]. Traversal takes $O(|C|)$ time. Share generation for $O(|C|)$ nodes: AND gates and OR gates cost $O(1)$, yielding $O(N^{5.2})$ total nodes and space. So, SS.Share time and space are $O(N^{5.2})$. SS.Combine processes input in $O(|S|)$ time. Valid share sets examined up to $\ell = O(N^{4.2})$, recursive algorithm takes $O(\ell^2)$ worst-case time. Secret reconstruction sums shares in $O(\ell)$ time with $O(\ell)$ space for valid sets. Therefore, SS.Combine time is $O(N^{8.4})$ worst-case, and space is $O(N^{4.2})$. Overall $\{0,1\}$ -LSSS time complexity is $O(N^{8.4})$, and space is $O(N^{5.2})$.

According to De Santis et al. (1990) [11], the computational complexities of the PZK protocol are as follows. The preprocessing algorithm $\text{PZK.Pre}(1^\lambda)$ requires $O(\lambda)$ time and space cost. The proof generation algorithm $\text{PZK.Prove}(\sigma_P, x, w)$ has time $O(\lambda \cdot |\Psi|)$ and space cost $O(\lambda + |\Psi|)$. Similarly, the verification algorithm $\text{PZK.Verify}(\sigma_V, x, \pi)$ exhibits time $O(\lambda \cdot |\Psi|)$ and space $O(\lambda + |\Psi|)$. Consequently, the overall time and space complexity of the PZK are $O(\lambda \cdot |\Psi|)$ and $O(\lambda + |\Psi|)$, respectively.

Based on Blum (1983) [13], $\text{C.Com}(m; r)$ takes $O(\lambda \cdot |m|)$ time and $O(\lambda + |m|)$ space. $\text{C.Verify}(m, \text{com}, r)$ takes $O(\lambda \cdot |m|)$ time and $O(\lambda + |m|)$ space. So, the commitment scheme's time and space costs are $O(\lambda \cdot |m|)$ and $O(\lambda + |m|)$.

From Gentry et al. [21], FHE.Setup takes $O(1)$ for parameter selection, $O(nm)$ for matrix $A \in \mathbb{Z}_q^{n \times m}$, $O(n)$ for secret key s and error e , and $O(n^2m)$ for public key

$\tilde{A} = [A; s^T A + ce^T]$ with $O(nm)$ space. Therefore, FHE.Setup time is $O(n^2m)$ and space is $O(nm)$. FHE.Encrypt takes $O(m^2)$ for random matrix $R \in \{0,1\}^{m \times m}$ and $O(nm^2)$ for ciphertext $ct = \tilde{A}R + \mu G$ (where μ is the message) with $O(nm)$ space. So, FHE.Encrypt time is $O(nm^2)$ and space is $O(nm)$. FHE.Eval takes $O(nm)$ for addition. Multiplication involves $O(m^2 \log q)$ for gadget decomposition G^{-1} and $O(nm^2)$ for matrix multiplication, yielding $O(2^d nm^2 \log q)$ for depth d circuits with $O(nm)$ space. Hence, FHE.Eval time is $O(2^d nm^2 \log q)$ and space is $O(nm)$. FHE.Decrypt takes $O(nm)$ for inner product $\langle sk, ct \cdot w_m \rangle$ and $O(1)$ for rounding with $O(1)$ space. Thus, FHE.Decrypt time is $O(nm)$ and space is $O(1)$. Overall, FHE time complexity is $O(2^d nm^2 \log q)$, and space is $O(nm)$.

Putting $\{0,1\}$ -LSSS and FHE together, the TFHE of Boneh et al. (2018) [4] consists of $O(2^d nm^2 \log q + N^{8.4})$ time, and $O(nm + N^{5.2})$ space complexity.

B. Universal Thresholdizer from $\{0,1\}$ -LSSS and FHE - Campanelli and Khoshakhlagh (2021) [6]

This method adapts TFHE from [4] directly, so we recall its cumulative complexity with $O(2^d nm^2 \log q + N^{8.4})$ for time and $O(nm + N^{5.2})$ for space from Section VI-A.

For HS, this technique uses a context-hiding homomorphic signature $\text{HS} = (\text{HS.Setup}, \text{HS.Sign}, \text{HS.Verify}, \text{HS.Eval})$ from [37], which is based on lattices.

- $\text{HS.Setup}(1^\lambda)$ generates lattice parameters ($t : O(\lambda)$) and performs SIS-based key generation ($O(\lambda^2)$ time, $O(\lambda^2)$ space for keys). Thus, HS.Setup time is $O(\lambda^2)$ and space is $O(\lambda^2)$.
- $\text{HS.Sign}(sk_{hs}, \tau, m)$ computes hash ($O(|m| + |\tau|)$ time), samples short vector ($O(\lambda^2)$ time), and performs matrix-vector multiplication ($O(\lambda^2)$ time), with $O(\lambda^2 + |m| + |\tau|)$ space. Therefore, HS.Sign time is $O(\lambda^2 + |m| + |\tau|)$ and space is $O(\lambda^2 + |m| + |\tau|)$.
- $\text{HS.Verify}(pk_{hs}, \tau, m, \sigma, f)$ computes hash ($O(|m| + |\tau|)$ time), verifies function evaluation ($O(\lambda|f|)$ time), and performs matrix-vector multiplication with norm check ($O(\lambda^2)$ time), with $O(\lambda^2 + |m| + |\tau| + |f|)$ space. Hence,

HS.Verify time is $O(\lambda^2 + \lambda|f| + |m| + |\tau|)$ and space is $O(\lambda^2 + |m| + |\tau| + |f|)$.

- HS.Eval(pk_{hs}, τ, f, σ) evaluates function on signatures ($O(\lambda|f|d)$ time) and performs matrix-vector operations ($O(\lambda^2|f|)$ time), with $O(\lambda^2 \log d + |\tau| + |f|)$ space. Thus, HS.Eval time is $O(\lambda|f|(\lambda + d))$ and space is $O(\lambda^2 \log d + |\tau| + |f|)$.
- Context-hiding simulator¹ requires ($O(\lambda^2 + |f|)$ time, $O(\lambda^2 + |f|)$ space), ensuring $\sigma_{f,y}$ reveals only $y = f(x)$.

C. Universal Thresholdizer from HSS and HS - Campanelli and Khoshakhlagh (2021) [6]

In this technique, they use the same HS as Figure 5, so we recall its total time and space complexity as $O(\lambda^2 \cdot |f| + \lambda \cdot |f| \cdot d)$ and $O(\lambda^2 \cdot \log d + |m| + |\tau| + |f|)$, respectively.

For HSS, they adapted the 2-party Homomorphic Secret Sharing from [23], and for the lattice-based instantiation, they refer to [39].

- HSS.Share($1^\lambda, x$) generates LWE-based random matrices of dimension $n \times m$ ($O(nm)$ time), samples error vectors ($O(n)$ time), and performs matrix operations to encode the secret ($O(n^2)$ time), with $O(nm)$ space for storing shares. Thus, HSS.Share time is $O(n^2 + nm)$ and space is $O(nm)$.
- HSS.Eval_b(share_b, C) for $b \in \{1, 2\}$ performs homomorphic operations on the share according to circuit C : addition operations ($O(n)$ time), multiplication ($O(n^2 \log q)$ time due to noise management), resulting in $O(2^d n^2 \log q)$ time for a circuit of depth d , with $O(n)$ space for the evaluation share. Therefore, HSS.Eval_b time complexity is $O(2^d n^2 \log q)$ and space is $O(n)$.
- HSS.Combine(y_1, y_2) performs bit-wise operations on the shares ($O(n)$ time) and final decoding ($O(1)$ time), with $O(1)$ space for the result. Thus, HSS.Combine time is $O(n)$ and space is $O(1)$.
- Overall, the HSS scheme has time complexity $O(2^d n^2 \log q)$ dominated by evaluation, and space complexity $O(nm)$ dominated by the shares.

Overall time complexity of HSS+HS Figure 6 is dominated by $O(\lambda \cdot |C| \cdot \text{poly}(\lambda))$ from PartEval and VfyEval operations, while space complexity includes $O(\lambda + |x|)$ for setup, $O(\lambda + |x| + \text{poly}(\lambda))$ for partial evaluation, $O(\lambda \cdot |C| + \text{poly}(\lambda))$ for verification, and $O(\text{poly}(\lambda) + |y|)$ for combine.

D. Universal Thresholdizer from $\{0, 1\}$ -LSSS and KHPRF - Ebrahimi et al. (2024) [5]

This type of UT, uses $\{0, 1\}$ -LSSS from [4], recalling its cumulative time cost as $O(N^{8.4})$, and space as $O(N^{5.2})$.

The FHE of [5] is an instantiation of [18] and [21] with the following complexity:

- FHE.Setup($1^\lambda, 1^d$) performs key generation and parameter selection ($O(\lambda \cdot d \cdot n^2 \cdot \log q)$ time), with $O(n \cdot \log q)$

space for storing pk and $sk \in \mathbb{Z}_q^n$. Thus, FHE.Setup time is $O(\lambda \cdot d \cdot n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.

- FHE.Encrypt(pk, μ) executes matrix-vector multiplications ($O(n^2 \cdot \log q)$ time), with $O(n \cdot \log q)$ space for ciphertext storage. Therefore, FHE.Encrypt time is $O(n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.
- FHE.Eval($pk, C, ct_1, \dots, ct_k$) processes circuit C on input ciphertexts ($O(|C| \cdot n^2 \cdot \log q)$ time), with $O(n \cdot \log q)$ space for the output ciphertext. Hence, FHE.Eval time is $O(|C| \cdot n^2 \cdot \log q)$ and space is $O(n \cdot \log q)$.
- FHE.Dec₀(sk, ct) computes inner products ($O(n \cdot \log q)$ time), with $O(\log q)$ space for the scalar output. Thus, FHE.Dec₀ time is $O(n \cdot \log q)$ and space is $O(\log q)$.
- FHE.Dec₁(p) performs threshold checking ($O(1)$ time), with $O(1)$ space for single-bit output. Therefore, FHE.Dec₁ time is $O(1)$ and space is $O(1)$.
- In total, the FHE scheme has time complexity $O(\lambda \cdot d \cdot n^2 \cdot \log q + |C| \cdot n^2 \cdot \log q)$ dominated by setup and evaluation, with space complexity $O(n \cdot \log q)$ for ciphertext and key storage.

This method uses KHPRF from [19]; KHPRF primitive is δ -almost key homomorphic where for any $k_1, k_2 \in K$, $F(k_1, x) + F(k_2, x) = F(k_1 + k_2, x) + e$, with $|e| \leq \delta$, with analysis spanning several functions. PRF evaluation function $F(k, x)$ requires $O(|k| + |x|)$ time complexity and $O(1)$ space complexity for output. Key generation for KHPRF consumes $O(\lambda)$ time complexity with $O(\lambda)$ space complexity for key storage. Secret sharing of KHPRF keys demands $O(n \cdot \ell)$ time complexity where n is the number of parties and ℓ the number of rows in the secret sharing matrix, with $O(n \cdot \ell)$ space complexity for all shares. Homomorphic evaluation functionality requires $O(L)$ time complexity, where L indicates the number of homomorphic operations, with $O(1)$ space complexity. Error handling mechanism performs constant time operations with $O(1)$ time complexity and $O(1)$ space complexity. Total KHPRF scheme exhibits time complexity $O(\lambda + n \cdot \ell + L + |k| + |x|)$, mostly dominated by secret sharing, with lattice-based instantiations requiring $O(n \cdot \ell \cdot \text{polylog}(q))$ where q is the lattice modulus. Total space complexity reaches $O(n \cdot \ell + \lambda)$, dominated by storage for key shares.

Combining $\{0, 1\}$ -LSSS with FHE and KHPRF, the TFHE scheme used in [5] exhibits time complexity $O(2^d \cdot n \cdot m^2 \cdot \log q \cdot |C| + N^{8.4})$ dominated by FHE evaluation and LSSS operations, with space complexity $O(n \cdot m + N^{5.2})$ dominated by FHE key storage and LSSS requirements.

Ebrahimi and Yadav (2024) [5] adapt PZK from [11], recalling the overall time and space complexity of the PZK as $O(\lambda \cdot |\Psi|)$ and $O(\lambda + |\Psi|)$, respectively.

Here, we consider the commitment scheme from [13], which is also mentioned in [4]. Since it is not mentioned in [5], it is unclear which scheme they exactly use and whether it is attributed to a specific sub-procedure. So, we consider the commitment scheme's time and space costs as $O(\lambda \cdot |m|)$ and $O(\lambda + |m|)$.

¹The context-hiding property ensures that a signature $\sigma_{f,y}$ reveals nothing about the input beyond the function output $y = f(x)$, which is essential for preserving privacy in the UT constructions described in the [6].

In this technique, the construction of TreeSSS = (TreeSSS.Share, TreeSSS.Combine) is inspired by Gupta and Mahajan's work (1996) [20] on constructing majority functions using small majority gates. TreeSSS.Share($sk, A_{N,t}$) implements tree secret sharing, taking $O((2s-1)^L s^2 \log q)$ time over L iterations, with $O((2s-1)^L \log q)$ space for level- L shares. Distributing shares takes $O((2s-1)^L)$ time and $O((2s-1)^L \log q/N)$ space per party, with $L \geq \log_{cs} N + \log_s N + O(1)$, $(2s-1)^L = O(N^{3+2.3/\log s})$. TreeSSS.Combine(B) reconstructs shares level by level, taking $O(L(2s-1)^L s^2 \log q)$ time and $O((2s-1)^L \log q)$ space; thus, total TreeSSS time cost is $O((2s-1)^L s^2 \log q) = O(N^{3+2.3/\log s} s^2 \log q)$, and space cost is $O((2s-1)^L \log q) = O(N^{3+2.3/\log s} \log q)$.

They use Brakerski et al. (2014) [12] as the basis for their LWE-based FHE = (FHE.Setup, FHE.KeyGen, FHE.Enc, FHE.Dec, FHE.Add, FHE.Mult, FHE.Refresh) implementation with bootstrapping.

- FHE.Setup($1^\lambda, 1^L, b$) creates $L+1$ parameter levels ($O((L+1) \cdot \text{poly}(\lambda, \log q))$ time/space). Thus, FHE.Setup time and space are both $O((L+1) \cdot \text{poly}(\lambda, \log q))$.
- FHE.KeyGen($\{params_j\}$) generates keys for all levels ($O(Ln^3 \log^2 q)$ time, $O(Ln^3 \log q)$ space). Therefore, FHE.KeyGen time is $O(Ln^3 \log^2 q)$ and space is $O(Ln^3 \log q)$.
- FHE.Enc($params, pk, m$) encrypts at highest level ($O(n^2 \log q)$ time, $O(n \log q)$ space). Hence, FHE.Enc time is $O(n^2 \log q)$ and space is $O(n \log q)$.
- FHE.Dec($params, sk, c$) decrypts ($O(n \log q)$ time, $O(n)$ space). Thus, FHE.Dec time is $O(n \log q)$ and space is $O(n)$.
- FHE.Add(pk, c_1, c_2) adds and refreshes ($O(n^3 \log^2 q)$ time, $O(n^2 \log q)$ space). Therefore, FHE.Add time is $O(n^3 \log^2 q)$ and space is $O(n^2 \log q)$.
- FHE.Mult(pk, c_1, c_2) multiplies and refreshes ($O(n^3 \log^2 q)$ time, $O(n^2 \log q)$ space). Hence, FHE.Mult time is $O(n^3 \log^2 q)$ and space is $O(n^2 \log q)$.
- FHE.Refresh(c, τ, q_j, q_{j-1}) consists of expanding ($O(n^2 \log^2 q)$ time), scaling ($O(n^2 \log q)$ time), SwitchKey ($O(n^3 \log^2 q)$ time), with $O(n^2 \log q)$ space. Thus, FHE.Refresh time is $O(n^3 \log^2 q)$ and space is $O(n^2 \log q)$.
- The commutative time and space complexity of FHE of this method (with bootstrapping) are $O(\lambda^4 \times |C|)$ and $O(\log \lambda \times \lambda^3)$, respectively.

Cheon et al.'s UT refers to the communication (not computational) efficiency of their construction compared to previous approaches by reducing the number of secret shares from Boneh et al.'s $O(N^{5.3})$ to approximately $O(N^3)$, Cheon et al. achieve efficiency gains for deployments. While both constructions have exponential offline verification costs, this is a one-time cost during setup.

To conclude our analysis on construction methods' efficiency from a time complexity perspective, the method by Boneh et al. [4] exhibits complexity of $O(N \cdot \lambda \cdot n \cdot \log q + |C| \cdot n^2 \cdot \log q)$, scaling linearly with the number of parties N . Cheon et al. [7] improves efficiency for large numbers of participants with complexity $O(N^{3+2.3/\log s} \cdot s^2 \cdot \log q + |F| \cdot n^2 \cdot \log q)$, which performs better than Boneh's approach when the parameter s is chosen optimally. In contrast, Ebrahimi and Yadav [5] demonstrate significantly higher complexity at $O(2^d \cdot n \cdot m^2 \cdot \log q \cdot |C| + N^{8.4})$, with exponential dependency on circuit depth d and polynomial scaling of $N^{8.4}$, making it less efficient for deeper circuits and larger committee sizes. The Campanelli and Khoshakhlagh [6] TFHE+HS construction achieves $O(N \cdot \ell \cdot n + |C| \cdot n^2 \cdot \log q)$, similar to Boneh's approach but with potential improvements in the circuit evaluation component. Finally, their HSS+HS approach for the restricted (2, 2)-threshold case demonstrates the most favorable time cost at $O(\lambda \cdot |C| \cdot \text{poly}(\lambda))$, though its applicability is limited to two-party settings.

Regarding space complexity, the techniques also exhibit differences in memory requirements. Boneh et al. [4] requires $O(N \cdot n \cdot \log q \cdot (\ell + \lambda))$ space, scaling linearly with the number of parties. Cheon et al. [7] demonstrates space cost of $O(N^{3+\frac{2.3}{\log s}} \cdot \log q + |S| \cdot N^{2+\frac{2.3}{\log s}} \cdot \log q)$, which grows faster with N but can be more efficient than Boneh's method for carefully chosen parameters. Ebrahimi and Yadav [5] exhibits space requirements of $O(N^{5.2} + n \cdot m + n \cdot (\lambda + |\Psi|))$, dominated by the $N^{5.2}$ term for large committees. The Campanelli and Khoshakhlagh [6] TFHE + HS approach uses $O(n \cdot \log q \cdot (N + |C|) + |y|)$ space, offering better scaling with N compared to Cheon's and Ebrahimi's methods. Their HSS+HS construction provides the most space-efficient solution at $O(\lambda \cdot |C| + |x| + |y| + \text{poly}(\lambda))$, though again limited to the two-party threshold setting, making it suitable only for specialized applications where minimizing memory usage is crucial and only two parties are involved.

REFERENCES

- [1] A. Shamir, "How to share a secret," *Commun. ACM*, vol. 22, no. 11, p. 612–613, Nov. 1979. [Online]. Available: <https://doi.org/10.1145/359168.359176>
- [2] G. R. BLAKLEY, "Safeguarding cryptographic keys," in 1979 *International Workshop on Managing Requirements Knowledge (MARK)*, 1979, pp. 313–318.
- [3] D. Boneh, R. Gennaro, S. Goldfeder, and S. Kim, "A lattice-based universal thresholdizer for cryptographic systems," *Cryptology ePrint Archive*, Paper 2017/251, 2017. [Online]. Available: <https://eprint.iacr.org/2017/251>
- [4] D. Boneh, R. Gennaro, S. Goldfeder, A. Jain, S. Kim, P. M. R. Rasmussen, and A. Sahai, "Threshold cryptosystems from threshold fully homomorphic encryption," in *Advances in Cryptology – CRYPTO 2018*, H. Shacham and A. Boldyreva, Eds. Springer International Publishing, 2018, pp. 565–596.
- [5] E. Ebrahimi and A. Yadav, "Strongly secure universal thresholdizer," in *Advances in Cryptology – ASIACRYPT 2024*, K.-M. Chung and Y. Sasaki, Eds. Springer Nature Singapore, 2025, pp. 207–239.
- [6] M. Campanelli and H. Khoshakhlagh, "Succinct publicly-certifiable proofs," in *Progress in Cryptology – INDOCRYPT 2021*, A. Adhikari, R. Küsters, and B. Preneel, Eds. Cham: Springer International Publishing, 2021, pp. 607–631.

- [7] J. H. Cheon, W. Cho, and J. Kim, "Improved universal thresholdizer from iterative shamir secret sharing," *Journal of Cryptology*, vol. 38, no. 15, 2025. [Online]. Available: <https://doi.org/10.1007/s00145-024-09536-z>
- [8] O. Regev, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, Sep. 2009. [Online]. Available: <https://doi.org/10.1145/1568318.1568324>
- [9] —, "On lattices, learning with errors, random linear codes, and cryptography," *J. ACM*, vol. 56, no. 6, Sep. 2009. [Online]. Available: <https://doi.org/10.1145/1568318.1568324>
- [10] M. Ajtai, "Generating hard instances of lattice problems," Weizmann, Report (TR96-007), 1996.
- [11] A. De Santis, S. Micali, and G. Persiano, "Non-interactive zero-knowledge with preprocessing," in *Advances in Cryptology — CRYPTO'88*, ser. Lecture Notes in Computer Science, S. Goldwasser, Ed., vol. 403. Springer, New York, NY, 1990.
- [12] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," *ACM Trans. Comput. Theory*, vol. 6, no. 3, Jul. 2014. [Online]. Available: <https://doi.org/10.1145/2633600>
- [13] M. Blum, "Coin flipping by telephone a protocol for solving impossible problems," *SIGACT News*, vol. 15, no. 1, p. 23–27, Jan. 1983. [Online]. Available: <https://doi.org/10.1145/1008908.1008911>
- [14] B. Barak, S.-J. Ong, and S. Vadhan, "Derandomization in cryptography," in *Advances in Cryptology - CRYPTO 2003*, ser. Lecture Notes in Computer Science, D. Boneh, Ed., vol. 2729. Springer, Berlin, Heidelberg, 2003. [Online]. Available: https://doi.org/10.1007/978-3-540-45146-4_18
- [15] R. Goyal, S. Hohenberger, V. Koppula, and B. Waters, "A generic approach to constructing and proving verifiable random functions," in *Theory of Cryptography. TCC 2017*, ser. Lecture Notes in Computer Science, Y. Kalai and L. Reyzin, Eds., vol. 10678. Springer, Cham, 2017.
- [16] I. Chillotti, N. Gama, M. Georgieva, and M. Izabachène, "Faster fully homomorphic encryption: Bootstrapping in less than 0.1 seconds," in *Advances in Cryptology – ASIACRYPT 2016*, ser. Lecture Notes in Computer Science, J. H. Cheon and T. Takagi, Eds., vol. 10031. Springer, Berlin, Heidelberg, 2016. [Online]. Available: https://doi.org/10.1007/978-3-662-53887-6_1
- [17] A. Jain, P. M. R. Rasmussen, and A. Sahai, "Threshold fully homomorphic encryption," Cryptology ePrint Archive, Paper 2017/257, 2017. [Online]. Available: <https://eprint.iacr.org/2017/257>
- [18] Z. Brakerski and V. Vaikuntanathan, "Efficient fully homomorphic encryption from (standard) lwe," in *2011 IEEE 52nd Annual Symposium on Foundations of Computer Science*, 2011, pp. 97–106.
- [19] D. Boneh, K. Lewi, H. Montgomery, and A. Raghunathan, "Key homomorphic prfs and their applications," in *Advances in Cryptology – CRYPTO 2013*, ser. Lecture Notes in Computer Science, R. Canetti and J. A. Garay, Eds., vol. 8042. Springer, Berlin, Heidelberg, 2013. [Online]. Available: https://doi.org/10.1007/978-3-642-40041-4_23
- [20] A. Gupta and S. Mahajan, "Using amplification to compute majority with small majority gates," *Comput. Complex.*, vol. 6, no. 1, p. 46–63, Jan. 1996. [Online]. Available: <https://doi.org/10.1007/BF01202041>
- [21] C. Gentry, A. Sahai, and B. Waters, "Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based," in *Advances in Cryptology – CRYPTO 2013*, ser. Lecture Notes in Computer Science, R. Canetti and J. A. Garay, Eds., vol. 8042. Springer, Berlin, Heidelberg, 2013.
- [22] D. Boneh, R. Gennaro, S. Goldfeder, and S. Kim, "A lattice-based universal thresholdizer for cryptographic systems," Cryptology ePrint Archive, Paper 2017/251, 2017. [Online]. Available: <https://eprint.iacr.org/2017/251>
- [23] E. Boyle, N. Gilboa, and Y. Ishai, "Breaking the circuit size barrier for secure computation under ddh," in *Advances in Cryptology – CRYPTO 2016*, ser. Lecture Notes in Computer Science, M. Robshaw and J. Katz, Eds., vol. 9814. Springer, Berlin, Heidelberg, 2016. [Online]. Available: https://doi.org/10.1007/978-3-662-53018-4_19
- [24] Y. Desmedt and Y. Frankel, "Threshold cryptosystems," in *Advances in Cryptology — CRYPTO'89 Proceedings*, ser. Lecture Notes in Computer Science, G. Brassard, Ed., vol. 435. Springer, New York, NY, 1990. [Online]. Available: https://doi.org/10.1007/0-387-34805-0_28
- [25] O. Goldreich, S. Goldwasser, and S. Micali, "How to construct random functions," *J. ACM*, vol. 33, no. 4, p. 792–807, Aug. 1986. [Online]. Available: <https://doi.org/10.1145/6490.6503>
- [26] R. Cramer, I. Damgård, and Y. Ishai, "Share conversion, pseudorandom secret-sharing and applications to secure computation," in *Proceedings of the Second International Conference on Theory of Cryptography*, ser. TCC'05. Berlin, Heidelberg: Springer-Verlag, 2005, p. 342–362. [Online]. Available: https://doi.org/10.1007/978-3-540-30576-7_19
- [27] D. Boneh and D. M. Freeman, "Homomorphic signatures for polynomial functions," in *Advances in Cryptology – EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, K. G. Paterson, Ed., vol. 6632. Springer, Berlin, Heidelberg, 2011.
- [28] A. Chailloux, D. F. Ciocan, I. Kerenidis, and S. Vadhan, "Interactive and noninteractive zero knowledge are equivalent in the help model," in *Theory of Cryptography. TCC 2008*, ser. Lecture Notes in Computer Science, R. Canetti, Ed., vol. 4948. Springer, Berlin, Heidelberg, 2008. [Online]. Available: https://doi.org/10.1007/978-3-540-78524-8_28
- [29] J. Groth, R. Ostrovsky, and A. Sahai, "Perfect non-interactive zero knowledge for np," in *Advances in Cryptology - EUROCRYPT 2006*, ser. Lecture Notes in Computer Science, S. Vaudenay, Ed., vol. 4004. Springer, Berlin, Heidelberg, 2006. [Online]. Available: https://doi.org/10.1007/11761679_21
- [30] B. Barak, O. Goldreich, S. Goldwasser, and Y. Lindell, "Resettable-sound zero-knowledge and its applications," in *Proceedings 42nd IEEE Symposium on Foundations of Computer Science*, 2001, pp. 116–125.
- [31] A. Lombardi, F. Ma, and N. Spooner, "Post-quantum zero knowledge, revisited or: How to do quantum rewinding undetectably," *2022 IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, pp. 851–859, 2022. [Online]. Available: <https://api.semanticscholar.org/CorpusID:244527506>
- [32] C. Baum, L. Braun, A. Munch-Hansen, and P. Scholl, "Moz $\mathbb{Z}_{2^k}$ arella: Efficient vector-ole and zero-knowledge proofs over $\mathbb{Z}_{2^k}$," in *Advances in Cryptology – CRYPTO 2022*, Y. Dodis and T. Shrimpton, Eds. Cham: Springer Nature Switzerland, 2022, pp. 329–358.
- [33] X. Tang, L. Shi, X. Wang, K. Charbonnet, S. Tang, and S. Sun, "Zero-knowledge proof vulnerability analysis and security auditing," Cryptology ePrint Archive, Paper 2024/514, 2024. [Online]. Available: <https://eprint.iacr.org/2024/514>
- [34] S. Kim and D. J. Wu, "Multi-theorem preprocessing nizks from lattices," in *Advances in Cryptology – CRYPTO 2018*, ser. Lecture Notes in Computer Science, H. Shacham and A. Boldyreva, Eds., vol. 10992. Springer, Cham, 2018. [Online]. Available: https://doi.org/10.1007/978-3-319-96881-0_25
- [35] S. Katsumata, R. Nishimaki, S. Yamada, and T. Yamakawa, "Designated verifier/prover and preprocessing nizks from diffie-hellman assumptions," in *Advances in Cryptology – EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, Y. Ishai and V. Rijmen, Eds., vol. 11477. Springer, Cham, 2019.
- [36] W. Cho, J. Kim, and C. Lee, "(in)security of threshold fully homomorphic encryption based on shamir secret sharing," Cryptology ePrint Archive, Paper 2024/1858, 2024. [Online]. Available: <https://eprint.iacr.org/2024/1858>
- [37] S. Gorbunov, V. Vaikuntanathan, and D. Wichs, "Leveled fully homomorphic signatures from standard lattices," in *Proceedings of the Forty-Seventh Annual ACM Symposium on Theory of Computing*, ser. STOC '15. New York, NY, USA: Association for Computing Machinery, 2015, p. 469–477. [Online]. Available: <https://doi.org/10.1145/2746539.2746576>
- [38] N. Attrapadung, B. Libert, and T. Peters, "Efficient completely context-hiding quotable and linearly homomorphic signatures," in *Public-Key Cryptography – PKC 2013*, ser. Lecture Notes in Computer Science, K. Kurosawa and G. Hanaoka, Eds., vol. 7778. Springer, Berlin, Heidelberg, 2013. [Online]. Available: https://doi.org/10.1007/978-3-642-36362-7_24
- [39] E. Boyle, L. Kohl, and P. Scholl, "Homomorphic secret sharing from lattices without fhe," in *Advances in Cryptology – EUROCRYPT 2019*, ser. Lecture Notes in Computer Science, Y. Ishai and V. Rijmen, Eds., vol. 11477. Springer, Cham, 2019. [Online]. Available: https://doi.org/10.1007/978-3-030-17656-3_1
- [40] M. Bellare, E. Crites, C. Komlo, M. Maller, S. Tessaro, and C. Zhu, "Better than advertised security for non-interactive threshold signatures," in *Advances in Cryptology – CRYPTO 2022*, ser. Lecture Notes in Computer Science, Y. Dodis and T. Shrimpton, Eds., vol. 13510. Springer, Cham, 2022. [Online]. Available: https://doi.org/10.1007/978-3-031-15985-5_18
- [41] M. Bellare, S. Tessaro, and C. Zhu, "Stronger security for non-interactive threshold signatures: BLS and FROST," Cryptology

ePrint Archive, Paper 2022/833, 2022. [Online]. Available: <https://eprint.iacr.org/2022/833>

- [42] S. Agrawal, D. Stehlé, and A. Yadav, “Round-Optimal Lattice-Based Threshold Signatures, Revisited,” in *49th International Colloquium on Automata, Languages, and Programming (ICALP 2022)*, ser. Leibniz International Proceedings in Informatics (LIPIcs), M. Bojańczyk, E. Merelli, and D. P. Woodruff, Eds., vol. 229. Dagstuhl, Germany: Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 2022, pp. 8:1–8:20. [Online]. Available: <https://drops.dagstuhl.de/entities/document/10.4230/LIPIcs.ICALP.2022.8>
- [43] S. Bai, T. Lepoint, A. Roux-Langlois, and et al., “Improved security proofs in lattice-based cryptography: Using the rényi divergence rather than the statistical distance,” *Journal of Cryptology*, vol. 31, pp. 610–640, 2018. [Online]. Available: <https://doi.org/10.1007/s00145-017-9265-9>
- [44] Z. Zhou, Y. Li, Y. Wang, Z. Yang, B. Zhang, C. Hong, T. Wei, and W. Chen, “ZHE: Efficient Zero-Knowledge Proofs for HE Evaluations,” in *2025 IEEE Symposium on Security and Privacy (SP)*. Los Alamitos, CA, USA: IEEE Computer Society, May 2025, pp. 3087–3105. [Online]. Available: <https://doi.ieeecomputersociety.org/10.1109/SP61157.2025.00199>
- [45] A. Lewko and B. Waters, “Decentralizing attribute-based encryption,” in *Advances in Cryptology – EUROCRYPT 2011*, ser. Lecture Notes in Computer Science, K. G. Paterson, Ed., vol. 6632. Springer, Berlin, Heidelberg, 2011. [Online]. Available: https://doi.org/10.1007/978-3-642-20465-4_31
- [46] O. Goldreich, “On (valiant’s) polynomial-size monotone formula for majority,” in *Computational Complexity and Property Testing*, ser. Lecture Notes in Computer Science, O. Goldreich, Ed. Springer, Cham, 2020, vol. 12050. [Online]. Available: https://doi.org/10.1007/978-3-030-43662-9_3

APPENDIX

OTHER BUILDING BLOCKS

Definition 8. A fully homomorphic encryption FHE scheme is a tuple of PPT algorithms $FHE = (\text{Setup}, \text{Encrypt}, \text{Eval}, \text{Decrypt})$ with the following properties:

- $(pk, sk) \leftarrow \text{Setup}(1^\lambda, 1^d)$: On input the security parameter λ , and a depth bound d , the setup algorithm outputs a key pair (pk, sk) .
- $ct \leftarrow \text{Encrypt}(pk, \mu)$: On input a public key pk , and a message $\mu \in \{0, 1\}$, the encryption algorithm outputs a ciphertext ct .
- $\hat{ct} \leftarrow \text{Eval}(pk, C, ct_1, \dots, ct_k)$: On input a public key pk , a circuit $C : \{0, 1\}^k \rightarrow \{0, 1\}$ of depth at most d , and a tuple of ciphertexts $ct_1, \dots, ct_k$, the evaluation algorithm outputs an evaluated ciphertext $\hat{ct}$.
- $\hat{\mu} \leftarrow \text{Decrypt}(pk, sk, \hat{ct})$: On input a public key pk , secret key sk , and a ciphertext $\hat{ct}$, the decryption algorithm outputs a message $\hat{\mu} \in \{0, 1, \perp\}$.

Homomorphic Secret Sharing (HSS)

Definition 9. A homomorphic secret sharing scheme for function family $\mathcal{F}$ consists of $HSS = (\text{Share}, \text{Eval}_1, \text{Eval}_2, \text{Combine})$.

- $(\text{share}_1, \text{share}_2) \leftarrow \text{Share}(1^\lambda, x)$ produces two secret shares share_1 and share_2 of a secret $x \in \{0, 1\}^*$, given the security parameter 1^λ .
- $y_b \leftarrow \text{Eval}_b(\text{share}_b, C)$: For $b \in \{1, 2\}$, party b produces an evaluation share y_b of the computation $C(x)$ using only their share share_b and circuit $C \in \mathcal{F}$.
- $y \leftarrow \text{Combine}(y_1, y_2)$. The combination algorithm takes evaluation shares y_1 and y_2 and outputs the final result $y \in \{0, 1\}^*$ such that $y = C(x)$.

ROBUSTNESS BY BONEH. ET AL. [4]

Definition 10 (Incomplete Robustness [4]). A UT is robust if for all λ , and depth bound d , the following holds. For any PPT adversary $\mathcal{A}$, the experiment $\text{Exp}_{\mathcal{A}, \text{UT}, \text{rb}}(1^\lambda, 1^d)$ in Figure 10 outputs 1 with negligible probability.

```


$$\text{Exp}_{\mathcal{A}, \text{UT}, \text{rb}}(1^\lambda, 1^d)$$



---


1 :  $(x, \mathbb{A}) \leftarrow \mathcal{A}(1^\lambda, 1^d)$ 
2 :  $(pp, s_1, \dots, s_N) \leftarrow \text{UT.Setup}(1^\lambda, 1^d, \mathbb{A}, x)$ 
3 :  $y_i^* \leftarrow \mathcal{A}(pp, s_1, \dots, s_N)$ 
4 :  $\text{return } y_i^* \neq \text{UT.Eval}(pp, s_i, C) \wedge \text{UT.Verify}(pp, y_i^*, C) = 1$ 

```

Fig. 10: Incomplete Robustness Experiment for UT

SECURITY AND ROBUSTNESS BY CAMPANELLI AND KHOSHAKHLAGH [6]

The security properties of UT as defined in [6] are as follows:

Definition 11 (Security). A UT scheme satisfies security with respect to circuit sampler D if for all $\lambda \in \mathbb{N}$, there exists a PPT algorithm $\mathcal{S} = (\mathcal{S}_S, \mathcal{S}_E)$ such that for any PPT adversary $\mathcal{A}$, the following experiments $\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{real}}(1^\lambda, d, N)$ and $\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{ideal}}(1^\lambda, d, N)$ are computationally indistinguishable:

```


$$\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{real}}(1^\lambda, d, N)$$



---


1 :  $x \leftarrow \mathcal{A}(1^\lambda, d, N)$ 
2 :  $(pp, sk_1, \dots, sk_N) \leftarrow \text{UT.Setup}(1^\lambda, d, N, x)$ 
3 :  $C \leftarrow \mathcal{A}(pp)$ , where  $|C| = d - 1$ 
4 :  $\{sk_i\}_{i \in C} \rightarrow \mathcal{A}$ 
5 :  $\forall C \in \mathcal{Q} : \{y_i \leftarrow \text{UT.Eval}(pp, sk_i, C)\}_{i \in [N]} \leftarrow \mathcal{O}_{\text{UT}}^{\text{real}}(C)$ 
6 :  $b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{UT}}^{\text{real}}}(pp, \{sk_i\}_{i \in C})$ 
7 : return  $b$ 

```

Fig. 11: Real-World Experiment for UT

```


$$\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{ideal}}(1^\lambda, d, N)$$



---


1 :  $x \leftarrow \mathcal{A}(1^\lambda, d, N)$ 
2 :  $(pp, sk_1, \dots, sk_N) \leftarrow \text{Sim}_S(1^\lambda, d, N)$ 
3 :  $C \leftarrow \mathcal{A}(pp)$ , where  $|C| = d - 1$ 
4 :  $\{sk_i\}_{i \in C} \rightarrow \mathcal{A}$ 
5 :  $\forall C \in \mathcal{Q} : \{y_i \leftarrow \text{Sim}_E(\text{trapd}, C, C(x))\}_{i \in [N]} \leftarrow \mathcal{O}_{\text{UT}}^{\text{ideal}}(C)$ 
6 :  $b \leftarrow \mathcal{A}^{\mathcal{O}_{\text{UT}}^{\text{ideal}}}(pp, \{sk_i\}_{i \in C})$ 
7 : return  $b$ 

```

Fig. 12: Ideal-World Experiment for UT

Definition 12 (Robustness). A UT scheme satisfies robustness if for all $\lambda \in \mathbb{N}$, for any PPT adversary $\mathcal{A}$, the probability

that the following experiment $\text{Expt}_{\mathcal{A}, \text{robust}}(1^\lambda)$ outputs 1 is negligible in λ :

$\text{Exp}_{\mathcal{A}, \text{UT}}^{\text{pe}}(1^\lambda, d, N)$
 1 : $x \leftarrow \mathcal{A}(1^\lambda)$
 2 : $(\text{pp}, \text{sk}_1, \dots, \text{sk}_N) \leftarrow \text{UT.Setup}(1^\lambda, d, N, x)$
 3 : $(\text{pp}, \text{sk}_1, \dots, \text{sk}_N) \rightarrow \mathcal{A}$
 4 : $\pi_i^* \leftarrow \mathcal{A}(\text{pp}, \text{sk}_1, \dots, \text{sk}_N)$
 5 : $\pi_i \leftarrow \text{UT.PartEval}(\text{pp}, \text{sk}_i, i, C)$
 6 : **return** $(\pi_i^* \neq \pi_i) \wedge (\text{UT.VfyEval}(\text{pp}, \pi_i^*, i, C) = 1)$

Fig. 13: Partial Evaluation for Robustness of UT

FORMALIZATION OF ATTACK ON THE σ -BASED METHODS

a) *Security Attack*: In Figure 14, the Attack-Security algorithm exploits the binding properties required for robustness to extract information about $\text{sk}_i[\text{fhe}]$, violating the security requirement of Definition 11. The attack begins by constructing a knowledge base $\mathcal{K}$ of constraints on $\text{sk}_i[\text{fhe}]$ through repeated oracle queries with strategically chosen circuits $\{C_t\}_{t=0}^\lambda$. For each query, SignatureStructure decomposes the resulting signature σ_i^t into components $\alpha_t.\text{metadata}$, $\alpha_t.\text{binding}$, and $\alpha_t.\text{transform}$ that reflect the structural properties of the homomorphic signature scheme. The ExtractBindingInfo procedure then derives constraints ϕ_t by analyzing homomorphic transformations and verification paths, expressed as relations $\text{Constraint}(y_i^t, \phi_t, \text{sk}_i[\text{fhe}])$ that must be satisfied for the signature to verify correctly. These constraints are systematically solved using SolveConstraints, which applies a bit-by-bit analysis to recover partial information about $\text{sk}_i[\text{fhe}]$ through deterministic solving of equations derived from the signature binding properties. This partial information, combined with the adversary's knowledge of $d-1$ corrupted shares $\{\text{sk}_j[\text{fhe}]\}_{j \in \mathcal{C}}$, allows reconstruction of information about the master secret sk_{fhe} using Shamir's secret sharing reconstruction techniques. Finally, ValidityCheck confirms the correctness of the extracted information by verifying consistency with observed partial decryptions, enabling the adversary to distinguish between real and ideal worlds with non-negligible probability.

b) *Robustness Attack*: In Figure 16, the Attack-Robustness algorithm exploits the simulatability properties required for security to forge valid signatures for incorrect partial decryptions, demonstrating the impossibility of achieving robustness when security is enforced. The attack begins by computing a legitimate partial evaluation (y_i, σ_i) and analyzing the signature structure through DecomposeSignature, which extracts components $\alpha.\text{base}$, $\alpha.\text{homomorphic}$, $\alpha.\text{checksum}$, and $\alpha.\text{metadata}$, along with an analysis of hiding properties $\alpha.\text{hidingLevel}$ that determine which components can be manipulated without compromising verification. The procedure GenerateFakeDecryption creates a counterfeit partial decryption $y_i^* = (u_i^*, v_i^*)$ by applying targeted perturbations Δ_u and Δ_v to the components of the legitimate decryption, ensuring the result differs measurably

from the correct value while maintaining plausibility. FindCompatibleShare solves for a hypothetical share $\hat{\text{sk}}_i[\text{fhe}]$ that would produce y_i^* under the partial decryption operation, establishing the feasibility of the forgery. The core of the attack is ForgeSignature, which systematically transforms the legitimate signature into a forged signature σ_i^* through a series of operations: IdentifyManipulableComponents determines which signature elements can be modified based on hiding level, TransformHomomorphicPart applies the necessary transformations to the homomorphic components to support the fake output, RecalculateChecksum updates integrity verification values, and EnforceVerifiability makes final adjustments to ensure the forged signature passes verification, exploiting the simulatability property required for security to create a valid but semantically incorrect signature.

Attack-Security(pp, $\mathcal{C}$, $\{\text{sk}_j\}_{j \in \mathcal{C}}$, i)

```

1:  $\mathcal{K} \leftarrow \emptyset$ 
2:  $C_0 \leftarrow \text{SampleCircuit}(1^\lambda)$ 
3:  $\text{ct}'_0 \leftarrow \text{TFHE.Eval}(\text{pk}_{\text{fhe}}, \text{ct}_x, C_0)$ 
4:  $\pi_i^0 = (y_i^0, \sigma_i^0) \leftarrow \mathcal{O}_{\text{eval}}(i, C_0)$ 
5:  $\alpha_0 \leftarrow \text{SignatureStructure}(\sigma_i^0)$ 
6:  $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{Constraint}(y_i^0, \alpha_0, \text{sk}_i[\text{fhe}])\}$ 
7: for  $t \in \{1, \dots, \lambda\}$  do
8:    $C_t \leftarrow \text{SampleCircuit}(t, \alpha_{t-1})$ 
9:    $\text{ct}'_t \leftarrow \text{TFHE.Eval}(\text{pk}_{\text{fhe}}, \text{ct}_x, C_t)$ 
10:   $\pi_i^t = (y_i^t, \sigma_i^t) \leftarrow \mathcal{O}_{\text{eval}}(i, C_t)$ 
11:   $\alpha_t \leftarrow \text{SignatureStructure}(\sigma_i^t)$ 
12:   $\phi_t \leftarrow \text{ExtractBindingInfo}(y_i^t, \sigma_i^t, \alpha_t, C_t)$ 
13:   $\mathcal{K} \leftarrow \mathcal{K} \cup \{\text{Constraint}(y_i^t, \phi_t, \text{sk}_i[\text{fhe}])\}$ 
14: endfor
15:  $\tilde{\text{sk}}_i[\text{fhe}] \leftarrow \text{SolveConstraints}(\mathcal{K})$ 
16:  $\mathcal{S} \leftarrow \{\text{sk}_j[\text{fhe}]\}_{j \in \mathcal{C}} \cup \{\tilde{\text{sk}}_i[\text{fhe}]\}$ 
17:  $\tilde{\text{sk}}_{\text{fhe}} \leftarrow \text{ReconstructSecret}(\mathcal{S}, d)$ 
18: if  $\text{ValidityCheck}(\tilde{\text{sk}}_{\text{fhe}}, \{y_i^t\}_{t=0}^\lambda, \{\text{ct}'_t\}_{t=0}^\lambda)$  then
19:   return 0
20: else
21:   return 1
22: endif

```

Fig. 14: Attack against UT's Security when Robustness is Enforced

SignatureStructure(σ_i) <pre> 1: $\alpha.\text{metadata} \leftarrow \sigma_i.\text{metadata}$ 2: $\alpha.\text{binding} \leftarrow \sigma_i.\text{binding}$ 3: $\alpha.\text{transform} \leftarrow \sigma_i.\text{transform}$ 4: return α </pre> SolveConstraints($\mathcal{K}$) <pre> 1: $\tilde{\text{sk}}_i[\text{fhe}] \leftarrow \vec{0}$ 2: for $j \in \{1, \dots, \text{sk}_i[\text{fhe}] \}$ do 3: $\mathcal{K}_j \leftarrow \{\text{constraints} \in \mathcal{K} \mid \text{constraints.affects}(j)\}$ 4: if DeterministicallySolvable($\mathcal{K}_j$) then 5: $\tilde{\text{sk}}_i[\text{fhe}][j] \leftarrow \text{SolveBit}(\mathcal{K}_j, j)$ 6: else 7: $\tilde{\text{sk}}_i[\text{fhe}][j] \leftarrow \perp$ 8: endif 9: endfor 10: return $\tilde{\text{sk}}_i[\text{fhe}]$ </pre>	ExtractBindingInfo(y_i, σ_i, α, C) <pre> 1: $\phi.\text{homomorphism} \leftarrow \text{ExtractHomomorphism}(\alpha.\text{transform}, C)$ 2: $\phi.\text{verification} \leftarrow \text{ExtractVerificationPath}(\alpha.\text{binding}, y_i)$ 3: $\phi.\text{constraints} \leftarrow \text{DeriveKeyConstraints}(\phi.\text{homomorphism}, \phi.\text{verification})$ 4: return ϕ </pre> ValidityCheck($\tilde{\text{sk}}_{\text{fhe}}, \{y_i^t\}_{t=0}^\lambda, \{\text{ct}'_t\}_{t=0}^\lambda$) <pre> 1: valid $\leftarrow$ true 2: for $t \in \{0, \dots, \lambda\}$ do 3: $\text{sk}'_i[\text{fhe}] \leftarrow \text{ShareGenerate}(\tilde{\text{sk}}_{\text{fhe}}, i, d)$ 4: $\tilde{y}_i^t \leftarrow \text{TFHE.PartDec}(\text{sk}'_i[\text{fhe}], \text{ct}'_t)$ 5: if $\tilde{y}_i^t \neq y_i^t$ and $\tilde{y}_i^t \neq \perp$ then 6: valid $\leftarrow$ false 7: break 8: endif 9: endfor 10: return valid </pre>
---	--

Fig. 15: Sub-procedures for UT's Security Attack

Attack-Robustness(pp, sk_i, C')

```

1 : ct' ← TFHE.Eval(pkfhe, ctx, C')
2 : yi ← TFHE.PartDec(ski[fhe], ct')
3 : σi ← HS.Eval(pkhs, "i", CDec, ski[hs])
   | CDec := TFHE.PartDec(·, ct')
4 : α ← DecomposeSignature(σi)
5 : yi* ← GenerateFakeDecryption(yi)
6 : ski[fhe] ← FindCompatibleShare(yi*, ct', CDec)
7 : σi* ← ForgeSignature(σi, α, yi, yi*, ski[hs])
8 : return (yi*, σi*)

```

Fig. 16: Attack against UT's Robustness when Security is Enforced

DecomposeSignature(σ_i) <pre> 1 : α.base ← σ_i.base 2 : α.homomorphic ← σ_i.homomorphic 3 : α.checksum ← σ_i.checksum 4 : α.metadata ← σ_i.metadata 5 : α.hidingLevel ← AnalyzeHidingProperties(σ_i) 6 : return α </pre>	GenerateFakeDecryption(y_i) <pre> 1 : (u_i, v_i) ← y_i 2 : u_i[*] ← u_i ⊕ Δ_u 3 : v_i[*] ← v_i ⊕ Δ_v 4 : y_i[*] ← (u_i[*], v_i[*]) 5 : return y_i[*] </pre>
FindCompatibleShare(y_i[*], ct', C_{Dec}) <pre> 1 : sk_i[fhe] ← SolveForShare(y_i[*], ct', C_{Dec}) 2 : return sk_i[fhe] </pre>	ForgeSignature(σ_i, α, y_i, y_i[*], sk_i[hs]) <pre> 1 : M ← IdentifyManipulableComponents(α) 2 : δ ← CalculateTransformation(y_i, y_i[*]) 3 : σ_i[*].base ← α.base 4 : σ_i[*].homomorphic ← TransformHomomorphicPart(α.homomorphic, δ) 5 : σ_i[*].checksum ← RecalculateChecksum(σ_i[*].base, σ_i[*].homomorphic) 6 : σ_i[*].metadata ← α.metadata 7 : σ_i[*] ← EnforceVerifiability(σ_i[*], y_i[*]) 8 : return σ_i[*] </pre>

Fig. 17: Sub-procedures for Robustness Attack (Part 1)

IdentifyManipulableComponents(α) <pre> 1 : M ← ∅ 2 : if α.hidingLevel ≥ FullyHiding then 3 : M ← M ∪ {homomorphic} 4 : endif 5 : if α.hidingLevel ≥ PartiallyHiding then 6 : M ← M ∪ {checksum} 7 : endif 8 : return M </pre>	TransformHomomorphicPart(homomorphic, δ) <pre> 1 : homomorphic' ← homomorphic 2 : for j ∈ {1, ..., homomorphic } do 3 : if IsOutputDependent(j, homomorphic) then 4 : homomorphic'[j] ← homomorphic[j] ⊕ ApplyDelta(δ, j) 5 : endif 6 : endfor 7 : return homomorphic' </pre>
RecalculateChecksum(base, homomorphic) <pre> 1 : checksum ← ComputeChecksum(base, homomorphic) 2 : return checksum </pre>	EnforceVerifiability(σ_i[*], y_i[*]) <pre> 1 : σ_i^{**} ← AdjustToPassVerification(σ_i[*], y_i[*]) 2 : return σ_i^{**} </pre>

Fig. 18: Sub-procedures for Robustness Attack (Part 2)